

گزارش فنی، ممیزی و راهنمای توسعه

سامانه پایش و کنترل اتاق آزمون

مبتنی بر STM32F107VCT6

نسخه سند: ۰.۱

تاریخ ممیزی: شهریور ۱۴۰۵

راهنمای استفاده از سند

این گزارش براساس خواندن همه فایل‌های سورس، تنظیمات STM32CubeMX، پروژه Keil MDK و خروجی ساخت موجود تدوین شده است. ادعاهای عملکردی که تنها با اتصال برد قابل اثبات‌اند، با عنوان «نیازمند آزمون سخت‌افزاری» مشخص شده‌اند. فایل تنظیمات ظاهری گزارش docs/report-config.tex است و هر فصل در پوشه docs/chapters به‌صورت مستقل نگهداری می‌شود.

فهرست مطالب

۱	دامنه، هدف و نمای کلی سامانه	۱
۱	۱.۱ کاربرد	۱
۱	۲.۱ حدود نسخه حاضر	۱
۱	۳.۱ نمای جریان داده	۱
۲	۴.۱ واحدهای مهم	۲
۳	۲ سخت افزار و تنظیمات مولد کد	۳
۳	۱.۲ پردازنده و کلاک	۳
۳	۲.۲ پیرامونی های فعال	۳
۴	۳.۲ پایه های کاربردی قطعی	۴
۵	۴.۲ ایمنی تولید مجدد	۵
۶	۳ معماری نرم افزار و مالکیت فایل ها	۶
۶	۱.۳ لایه ها	۶
۶	۲.۳ قرارداد با فایل های تولیدی	۶
۷	۳.۳ زمان بندی تعاونی	۷
۷	۴.۳ همروندی و انتشار داده	۷
۷	۵.۳ حافظه پایدار	۷
۸	۴ درایورها، ماژول ها و وضعیت استفاده	۸

۸		دراپور دمای دیجیتال	۱.۴
۹		اندازه‌گیری انرژی	۲.۴
۹		رطوبت آنالوگ	۳.۴
۹		حسگر رطوبت دیجیتال	۴.۴
۹		فیلتر میانگین متحرک	۵.۴
۱۰		دراپورهای HAL	۶.۴
۱۱		۵ رفتار زمان اجرا و کنترل محیط	
۱۱		راه‌اندازی	۱.۵
۱۱		کنترل رطوبت	۲.۵
۱۱		کنترل سرمایش	۳.۵
۱۲		کنترل گرمایش	۴.۵
۱۲		حالت‌ها و ایمنی	۵.۵
۱۳		۶ پروتکل سریال رایانه	
۱۳		لایه انتقال	۱.۶
۱۳		فرمان‌ها	۲.۶
۱۴		قاب داده ۲۰۹ بایتی	۳.۶
۱۶		۷ نتیجه ممیزی و تغییرات انجام‌شده	
۱۶		نقاط قوت نسخه اولیه	۱.۷
۱۶		اشکال‌های بحرانی اصلاح‌شده	۲.۷
۱۷		موارد حفظ‌شده	۳.۷
۱۷		ریسک‌های باز	۴.۷
۱۷		چرا بعضی فایل‌ها حذف نشدند	۵.۷

۱۸	۸	ساخت، پروگرام و تولید مجدد
۱۸	۱.۸	ساخت با Keil
۱۸	۲.۸	ساخت با PlatformIO
۱۹	۳.۸	تولید مجدد با CubeMX
۱۹	۴.۸	ساخت گزارش
۲۰	۹	تأیید، تحویل و نگهداری آینده
۲۰	۱.۹	آزمون رومیزی بدون بار قدرت
۲۰	۲.۹	آزمون پروتکل
۲۰	۳.۹	آزمون کنترل
۲۱	۴.۹	معیار پذیرش نسخه
۲۱	۵.۹	اولویتهای توسعه بعدی
۲۱	۶.۹	قاعده نگهداری سند
۲۲	۱۰	مرجع کامل توابع لایه کاربرد
۲۲	۱.۱۰	حالت عمومی قابل مشاهده در دیباگر
۲۳	۲.۱۰	توابع اصلی چرخه اجرا
۲۴	۱.۲.۱۰	تابع App_Init
۲۴	۲.۲.۱۰	تابع App_Process
۲۴	۳.۱۰	ضرایب پیش فرض و بازنشانی
۲۴	۱.۳.۱۰	تابع App_ResetCalibrationToDefaults
۲۵	۴.۱۰	نصب و محاسبه کالیبراسیون فاز
۲۵	۱.۴.۱۰	تابع App_SetVoltageCalibration
۲۵	۲.۴.۱۰	تابع App_SetPhaseCalibration
۲۵	۳.۴.۱۰	تابع App_CalibratePhaseMultiPoint

۵.۱۰	کالیبراسیون جریان نول	۲۵
۱.۵.۱۰	تابع App_SetNeutralCurrentCalibration	۲۵
۲.۵.۱۰	تابع App_CalibrateNeutralCurrentMultiPoint	۲۶
۶.۱۰	جهت ترانس جریان و واحد نمایش	۲۶
۱.۶.۱۰	تابع App_SetCurrentPolarity	۲۶
۲.۶.۱۰	تابع App_SetDisplayUnits	۲۶
۷.۱۰	توابع داخلی مهم	۲۶
۸.۱۰	الگوی استفاده تک فاز و سه فاز	۲۷
۹.۱۰	چک لیست استفاده صحیح	۲۷
۱۱	مرجع کامل توابع درایورها	۲۸
۱.۱۱	ذخیره شناسه حسگر	۲۸
۱.۱.۱۱	تابع crc8_maxim	۲۸
۲.۱.۱۱	تابع crc32_record	۲۸
۳.۱.۱۱	تابع SensorAddressStore_IsValidRom	۲۸
۴.۱.۱۱	تابع table_is_valid	۲۹
۵.۱.۱۱	تابع SensorAddressStore_Load	۲۹
۶.۱.۱۱	تابع SensorAddressStore_Save	۲۹
۲.۱۱	درایور ADE7880	۲۹
۱.۲.۱۱	تابع ADE7880_ReadRegister	۲۹
۲.۲.۱۱	تابع ADE7880_WriteRegister	۳۰
۳.۲.۱۱	تابع ADE7880_ConfigSPI	۳۰
۴.۲.۱۱	تابع ADE7880_Run	۳۰
۵.۲.۱۱	تابع ADE7880_ReadVRMS	۳۰
۶.۲.۱۱	تابع ADE7880_ReadIRMS	۳۰

۳۰		ADE7880_ReadPowerz	تابع	۷.۲.۱۱
۳۱		ADE7880_ReadFPower	تابع	۸.۲.۱۱
۳۱		ADE7880_Reset	تابع	۹.۲.۱۱
۳۱		DS18B20 و OneWire	پشته	۳.۱۱
۳۱		OneWire	توابع پایه	۱.۳.۱۱
۳۱		جست و جوی شناسه		۲.۳.۱۱
۳۱		DS18B20	فرمان های	۳.۳.۱۱
۳۲		مدیر چندگذرگاهی		۴.۳.۱۱
۳۲		AM2315	درایور	۴.۱۱
۳۲		am2315_crc16	تابع	۱.۴.۱۱
۳۲		TEMP_HUM	تابع	۲.۴.۱۱
۳۳		MAF	فیلتر	۵.۱۱
۳۳		MAF_Init	تابع	۱.۵.۱۱
۳۳		MAF_Update	تابع	۲.۵.۱۱
۳۳		Average	تابع	۳.۵.۱۱
۳۴		۱۲ شرح جزئی بازآرایی و مرز تغییرات		
۳۴		وضعیت پیش از بازآرایی		۱.۱۲
۳۴		انتقال مالکیت		۲.۱۲
۳۴		تبدیل وقفه به کار معوق		۳.۱۲
۳۵		فشرده سازی قاب مخابراتی		۴.۱۲
۳۵		بازطراحی دریافت سریال		۵.۱۲
۳۵		بازطراحی حافظه شناسه ها		۶.۱۲
۳۵		اصلاح رفتار کنترل		۷.۱۲
۳۶		CubeMX	مرزهای حفاظت شده	۸.۱۲

۳۶	۹.۱۲	مواردی که عمداً تغییر نکردند
۳۶	۱۰.۱۲	موارد باقیمانده برای مرحله بعد
۳۷	۱۳	مرجع توابع تولیدشده و راهاندازی سختافزار
۳۷	۱.۱۳	فایل اصلی
۳۷	۱.۱.۱۳	تابع main
۳۷	۲.۱.۱۳	تابع SystemClock_Config
۳۸	۳.۱.۱۳	تابع MX_ADC1_Init
۳۸	۴.۱.۱۳	تابع MX_I2C1_Init
۳۸	۵.۱.۱۳	تابع MX_SPI2_Init
۳۸	۶.۱.۱۳	توابع MX_TIM1_Init، MX_TIM2_Init و MX_TIM3_Init
۳۸	۷.۱.۱۳	تابع MX_USART2_UART_Init
۳۸	۸.۱.۱۳	تابع MX_DMA_Init
۳۹	۹.۱.۱۳	تابع MX_GPIO_Init
۳۹	۱۰.۱.۱۳	تابع Error_Handler
۳۹	۲.۱۳	توابع پشتیبانی پیرامونی
۳۹	۱.۲.۱۳	تابع HAL_MspInit
۳۹	۲.۲.۱۳	توابع HAL_ADC_MspInit و HAL_ADC_MspDeInit
۳۹	۳.۲.۱۳	توابع HAL_I2C_MspInit و HAL_I2C_MspDeInit
۴۰	۴.۲.۱۳	توابع HAL_SPI_MspInit و HAL_SPI_MspDeInit
۴۰	۵.۲.۱۳	توابع پشتیبانی تایمرها
۴۰	۶.۲.۱۳	توابع HAL_UART_MspInit و HAL_UART_MspDeInit
۴۰	۳.۱۳	هندلرهای استثنا و وقفه
۴۰	۴.۱۳	فایل سامانه و راهانداز

۴۱	۱۴ نشانگر زنده بودن و آزمون دو چراغ
۴۱	۱.۱۴ هدف آزمون
۴۱	۲.۱۴ اتصال الکتریکی و قطب فعال
۴۲	۳.۱۴ تنظیمات مولد کد
۴۲	۴.۱۴ فایل ها و رابط عمومی
۴۲	۱.۴.۱۴ تابع DiagnosticLed_Init
۴۲	۲.۴.۱۴ تابع DiagnosticLed_SetMode
۴۲	۳.۴.۱۴ تابع DiagnosticLed_Process
۴۳	۴.۴.۱۴ تابع DiagnosticLed_GetMode
۴۳	۵.۱۴ الگوهای قابل انتخاب
۴۳	۱.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_OFF
۴۳	۲.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_HEARTBEAT
۴۳	۳.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_DANCE
۴۳	۴.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_ERROR
۴۳	۶.۱۴ اتصال به چرخه برنامه
۴۴	۷.۱۴ پروگرام با رابط چهار سیمه
۴۴	۸.۱۴ نتیجه پروگرام انجام شده
۴۴	۹.۱۴ روند آزمون و تفسیر نتیجه
۴۶	۱۵ راه اندازی ارتباط و اندازه گیری با ADE7880
۴۶	۱.۱۵ دامنه این مرحله و نتیجه قابل انتظار
۴۶	۲.۱۵ منبع بازیابی اتصال های قبلی
۴۷	۳.۱۵ کنترل الزامات دیتاشیت
۴۸	۴.۱۵ تنظیم از صفر در STM32CubeMX
۴۹	۵.۱۵ معماری کتابخانه قابل استفاده مجدد

۴۹	۶.۱۵ قرارداد کامنت گذاری و روش خواندن کد
۵۰	۷.۱۵ مرجع سریع توابع کتابخانه
۵۱	۸.۱۵ توالی راه اندازی برنامه
۵۱	۹.۱۵ رجیسترها و قالب داده خوانده شده
۵۲	۱۰.۱۵ خواندن در VS Code و PlatformIO
۵۳	۱۱.۱۵ ساخت و دیباگ در Keil MDK
۵۴	۱۲.۱۵ کالیبراسیون دو نقطه ای
۵۴	۱.۱۲.۱۵ ضرایب ولتاژ بازیابی شده و خروجی فعلی
۵۵	۲.۱۲.۱۵ مشاهده جریان، توان و ضریب توان در دیباگر
۵۷	۳.۱۲.۱۵ جهت CT، علامت توان و علامت ضریب توان
۵۸	۴.۱۲.۱۵ واحد مهندسی و نمایش سازگار با برنامه قدیمی
۵۹	۵.۱۲.۱۵ جدول واحد ضرایب پیش فرض هر سه فاز
۶۰	۶.۱۲.۱۵ استفاده تک فاز، دو کانال مستقل و سه فاز
۶۰	۷.۱۲.۱۵ تصمیم کالیبراسیون: یک خط چند نقطه ای، نه ضرایب قطعه ای
۶۱	۸.۱۲.۱۵ انتخاب نقاط و برداشت داده
۶۱	۹.۱۲.۱۵ کد کالیبراسیون ولتاژ، جریان و توان
۶۳	۱۳.۱۵ آزمون سخت افزاری مرحله به مرحله
۶۳	۱.۱۳.۱۵ آزمون بدون اعمال ولتاژ اندازه گیری
۶۴	۲.۱۳.۱۵ آزمون با منبع مرجع
۶۴	۱۴.۱۵ راهنمای عیب یابی
۶۵	۱۵.۱۵ نتیجه اعتبار سنجی نرم افزاری این مرحله
۶۶	۱۶.۱۵ چک لیست پذیرش این قابلیت
۶۷	۱۶ زیر سامانه DS18B20 با شماره گذاری پایدار
۶۷	۱.۱۶ نیازمندی و مسئله ترتیب

۶۸	۲.۱۶ اتصال الکتریکی و فرض‌های این نسخه
۶۸	۳.۱۶ معماری قابل انتقال
۶۹	۴.۱۶ جست‌وجو و تضمین صحت داده
۶۹	۵.۱۶ ماشین حالت و زمان‌بندی
۶۹	۶.۱۶ ذخیره نگاشت در Flash
۷۰	۷.۱۶ معرفی اولیه سنسورها
۷۱	۸.۱۶ تعویض سنسور و مرز تصمیم خودکار
۷۱	۹.۱۶ مشاهده در دیباگر
۷۱	۱۰.۱۶ حفاظت در برابر تولید مجدد CubeMX
۷۲	۱۱.۱۶ چک‌لیست آزمون روی برد
۷۲	۱۲.۱۶ محدودیت‌های اعلام‌شده

۱۷ نمایشگر کاراکتری و کلیدهای منو

۷۳	۱.۱۷ هدف و نتیجه این مرحله
۷۳	۲.۱۷ مبنای دیتاشیت و محدودیت الکتریکی مهم
۷۴	۳.۱۷ جدول پایه‌ها و تنظیم CubeMX
۷۵	۴.۱۷ معماری نرم‌افزار
۷۵	۵.۱۷ راه‌اندازی چهار بیتی و زمان‌بندی
۷۵	۶.۱۷ مرجع توابع کتابخانه
۷۷	۷.۱۷ وقفه کوتاه و حذف جهش کلید
۷۷	۸.۱۷ حفظ LED و کارهای هم‌زمان
۷۷	۹.۱۷ انتقال به میکروکنترلر دیگر
۷۸	۱۰.۱۷ حفاظت در برابر Generate Code
۷۸	۱۱.۱۷ روش آزمون روی برد
۷۹	۱۲.۱۷ عیب‌یابی و اعتبارسنجی

فصل ۱

دامنه، هدف و نمای کلی سامانه

۱.۱ کاربرد

برد کنترل برای یک اتاق آزمون تجهیزات برودتی ساخته شده است. دو ایستگاه اندازه‌گیری، در مجموع ۳۲ حسگر دمای دیجیتال را می‌خوانند؛ دو حسگر دمای محیط روی دو گذرگاه مستقل نیز مرجع کنترل اتاق‌اند. رطوبت محیط از خروجی آنالوگ حسگر خانواده HIH نمونه‌برداری می‌شود. مدار ADE7880 کمیت‌های الکتریکی دو کانال فعال را اندازه می‌گیرد. رایانه از طریق RS-232 داده‌ها را دریافت و رله‌ها، سرمایش، گرمایش، رطوبت‌ساز و نقطه‌های تنظیم را کنترل می‌کند.

۲.۱ حدود نسخه حاضر

این نسخه قابلیت تازه‌ای به محصول اضافه نمی‌کند. هدف آن حفظ رفتار شناخته‌شده، حذف خطرهای قطعی، جداسازی کد کاربردی از کد مولد، فراهم کردن ساخت با دو زنجیره ابزار و ایجاد مرجع نگهداری است. دو ایستگاه سوم و چهارم در قالب مخابراتی رزرو شده‌اند، ولی سخت‌افزار و منطق فعال ندارند.

۳.۱ نمای جریان داده

ترتیب منطقی اجرا چنین است:

۱. وقفه‌های تایمر تنها موعدهای اجرای کار را ثبت می‌کنند.
۲. حلقه اصلی کمیت الکتریکی، رطوبت و دما را در موعدهای مربوط می‌خواند.

۳. کنترل خودکار با هیستریزیس و پنجره زمانی هیتر خروجی ها را تنظیم می کند.
۴. یک قاب ثابت ۲۰۹ بایتی ساخته و روی درگاه سریال ارسال می شود.
۵. دریافت سریال به صورت وقفه ای و تک بایتی انجام و فرمان کامل در حلقه اصلی تفسیر می شود.

۴.۱ واحدهای مهم

دمای محیط در داخل برنامه با واحد یکدهم درجه سلسیوس نگهداری می شود. داده ۳۲ حسگر ایستگاه ها برای سازگاری با نرم افزار قبلی به شکل مقدار خام حسگر، یعنی واحد یک شانزدهم درجه، ارسال می شود. رطوبت ارسالی یکدهم درصد و مقادیر چندبایتی همگی کم ارزش ترین بایت در ابتدا هستند.

فصل ۲

سخت‌افزار و تنظیمات مولد کد

هشدار: جدول کامل پیرامونی‌های این فصل معماری محصول پیش از پاک‌سازی را ثبت می‌کند. در snapshot فعال این مرحله فقط کلاک سامانه، SWD، دو LED و SPI2/ADE7880 دوباره فعال شده‌اند؛ مراحل دقیق بازسازی فعلی در فصل ۱۵ است.

۱.۲ پردازنده و کلاک

قطعه انتخاب‌شده STM32F107VCT6 در بسته LQFP100 است. نوسان‌ساز خارجی ۸ مگاهرتز با ضریب ۹، کلاک سامانه ۷۲ مگاهرتز می‌سازد. کلاک گذرگاه اول ۳۶ و گذرگاه دوم ۷۲ مگاهرتز است. کلاک مبدل آنالوگ ۱۲ مگاهرتز تنظیم شده است.

۲.۲ پیرامونی‌های فعال

پیرامونی	تنظیم و کاربرد
ADC1	کانال‌های ۵ و ۴، اسکن پیوسته، زمان نمونه‌برداری ۵.۲۳۹ سیکل و انتقال چرخشی با کانال اول DMA1؛ کانال نخست منبع فعال رطوبت است.
SPI2	حالت ارباب، هشت‌بیتی، قطبیت بالا، لبه دوم، تقسیم کلاک ۶۴؛ ارتباط با ADE7880 و انتخاب تراشه روی پایه PB12.

تنظیم و کاربرد	پیرامونی
نرخ ۹۶۰۰، هشت بیت، یک توقف، بدون توازن، ارسال و دریافت؛ پایه‌های بازنگاشت شده PD5/PD6.	USART2
نرخ ۱۰۰ کیلوهرتز و بازنگاشت روی PB8/PB9؛ برای کتابخانه حسگر AM2315 آماده است ولی در مسیر فعال استفاده نمی‌شود.	I2C1
دوره یک ثانیه؛ موعد ارسال داده.	TIM1
دوره نیم ثانیه؛ موعد انرژی، رطوبت، شمارنده استارت و پنجره هیتر.	TIM2
دوره یک ثانیه؛ موعد خواندن حسگرهای دما.	TIM3

۳.۲ پایه‌های کاربردی قطعی

پایه	نقش	نکته
PB10	گذرگاه نخست 1-Wire	شانزده حسگر ایستگاه اول و حسگر محیط اول
PC7	گذرگاه دوم 1-Wire	شانزده حسگر ایستگاه دوم و حسگر محیط دوم
PA5	ورودی آنالوگ نخست	ورودی فعال محاسبه رطوبت
PA4	ورودی آنالوگ دوم	نمونه‌برداری می‌شود ولی فعلاً مصرف کاربردی ندارد
PE1	رطوبت‌ساز	فرمان فعال پایین
PE2	رله ایستگاه اول	فرمان دستی فعال پایین
PE4,PE5	دو خروجی سرمایش	هم‌زمان کنترل می‌شوند و فعال پایین‌اند
PB3	هیتر یا SSR	فعال پایین و دارای کنترل پنجره زمانی
PB12	انتخاب تراشه انرژی	در حالت بی‌کار بالا
PA13,PA14	برنامه‌ریزی و اشکال‌زدایی	رابط دو سیمه SWD برای J-Link؛ زمین و تغذیه مرجع نیز لازم است

پایه	نقش	نکته
PC13, PC14	دو چراغ عیب‌یابی	خروجی پوش‌پول کم‌سرعت، سطح اولیه بالا و الگوی غیرمسدودکننده زنده‌بودن برنامه

پایه‌های خروجی دیگر شامل PE3, PE6, PB4, PD0, PD2, PD3, PD8, PD14, PD15, PC8 در فایل مولد تعریف شده‌اند اما نقش دقیقشان از سورس معلوم نیست. دو پایه PE6 و PD3 مطابق رفتار نسخه قدیمی هنگام توقف کنترل در سطح بالا قرار می‌گیرند. پیش از حذف یا تغییر هر کدام باید شماتیک بررسی شود.

۴.۲ ایمنی تولید مجدد

گزینه نگهداشتن کد کاربر فعال، پشتیبان‌گیری نسخه قبلی فعال و حذف پیشین غیرفعال شده است. مقدار دوره تایمر دوم نیز از اختلاف خطرناک ۳۵ به مقدار عملیاتی ۳۵۹۹ همگام شد. پس از هر تولید مجدد باید تفاوت فایل‌ها بازبینی و وجود فراخوانی‌های App_Init و App_Process در بلوک‌های حفاظت‌شده کنترل شود.

فصل ۳

معماری نرم افزار و مالکیت فایل ها

۱.۳ لایه ها

فایل `Core/Src/main.c` راه اندازی تولید شده و دو فراخوانی `App_Init` و `App_Process` را نگه می دارد. فایل `Core/Src/stm32f1xx\_it.c` فقط هندلرهای استاندارد HAL را اجرا می کند. منطق فعال محصول در `Application/Src/app.c` قرار دارد. ماژول چراغ در همان لایه کاربرد و کتابخانه قابل حمل انرژی در `Libraries/ADE7880/Src/ade7880.c` جدا شده است.

کتابخانه های حسگر، میانگین گیری و ذخیره سازی قدیمی در پوشه `Libraries` حفظ شده اند، ولی در نسخه تمیز فعلی هنوز به چرخه برنامه متصل نیستند. وجود آن ها در پروژه ساخت به معنی فعال بودنشان در زمان اجرا نیست.

۲.۳ قرارداد با فایل های تولیدی

تنها تغییرات دستی مجاز در فایل مولد باید بین نشانگرهای `USER CODE` و `USER CODE BEGIN` باشد. در نسخه حاضر، سرآیند کاربرد در بلوک شامل ها، فراخوانی راه اندازی در بلوک شماره ۲ و فراخوانی پردازش در حلقه حفاظت شده قرار دارند. منطق کاربرد نباید به فایل وقفه منتقل شود.

تنظیم پایه `ADE_CS=PB12` و رابط `SPI2` در فایل `STM32F107.ioc` ثبت شده اند. پس از هر تولید مجدد باید تفاوت فایل ها در گیت، حالت ۳ رابط، کلاک مجاز و باقی ماندن بلوک های کاربر کنترل شوند.

۳.۳ زمان بندی تعاونی

نسخه فعال بدون تایمر کاربردی و بدون سیستم عامل کار می کند. تابع App_Process با HAL_GetTick موعدهای یک ثانیه ای نمونه و دوتاییه ای بازیابی ارتباط را کنترل می کند. اگر موعد نرسیده باشد بلافاصله بازمی گردد؛ بنابراین حلقه عادی تأخیر مسدودکننده ندارد.

ماژول چراغ نیز با همان الگوی تعاونی سرویس می شود. ادامه منظم الگو نشان می دهد حلقه اصلی هنوز در حال اجراست. خطای ارتباط ADE7880 الگوی سریع خطا را فعال می کند و اتصال موفق دوباره حالت عادی را برمی گرداند.

۴.۳ همروندی و انتشار داده

در نسخه فعلی هیچ حالت مشترک میان وقفه و لایه کاربرد وجود ندارد. هندلرهای تولیدی فقط سرویس استاندارد HAL را اجرا می کنند. حالت عمومی الکتریکی برای مشاهده دیباگر volatile است، ولی تمام تغییرهای آن در متن حلقه اصلی انجام می شود.

هر نمونه کامل ابتدا در متغیر محلی ساخته می شود. فقط پس از موفقیت تمام خواندن های A/B/C و نول، مقدارهای خام و مهندسی با هم منتشر می شوند؛ در نتیجه مصرف کننده داده نیمه تازه دریافت نمی کند.

۵.۳ حافظه پایدار

آخرین صفحه دو کیلوبایتی Flash از نشانی 0x0803F800 طبق طراحی قبلی برای جدول شناسه حسگر رزرو و از ناحیه لینک هر دو زنجیره ساخت حذف شده است. ماژول ذخیره سازی در مخزن باقی مانده، اما تا زمان بازگرداندن سنسورها از لایه کاربرد فعال فراخوانی نمی شود.

ضرایب کالیبراسیون ADE7880 فعلاً در RAM هستند. جدول پیش فرض با هر بازنشانی بازمی گردد. ضرایب تأیید شده برای ماندگاری باید در مرحله بعد در کد ثابت یا حافظه غیرفرار ذخیره و هنگام راه اندازی نصب شوند.

فصل ۴

دراپورها، ماژول‌ها و وضعیت استفاده

۱.۴ دراپور دمای دیجیتال

زیرسامانه فعال دما چهار بخش دارد. کتابخانه Libraries/OneWire زمان‌بندی استاندارد، CRC8 و جست‌وجوی ROM را مستقل از میکروکنترلر انجام می‌دهد؛ Libraries/DS18B20 فرمان‌های حسگر، تبدیل مقدار علامت‌دار و اعتبارسنجی حافظه موقت را فراهم می‌کند؛ مدیر Libraries/DS18B20Manager کشف چند گذرگاه، شماره‌گذاری پایدار و نمونه‌برداری بدون انتظار طولانی را بر عهده دارد؛ و Libraries/SensorAddressStore تنها لایه وابسته به STM32 HAL برای ذخیره نگاشت در Flash است.

گذرگاه صفر روی PB10 و گذرگاه یک روی PC7 قرار دارد. هر دو پایه در فایل CubeMX به شکل خروجی تخلیه‌باز ثبت شده‌اند و به مقاومت بالاکش خارجی نیاز دارند. تبدیل همه حسگرهای هر گذرگاه با فرمان سراسری آغاز می‌شود، انتظار ۷۵۰ میلی‌ثانیه‌ای در ماشین حالت انجام می‌گیرد و هر حافظه موقت با Match ROM و کنترل CRC خوانده می‌شود.

سنسور تازه خودکار به اولین شماره خالی متصل می‌شود. مقدار و اعتبار به ترتیب در دو آرایه زیر منتشر می‌شوند:

g_temperature_c ○

g_temperature_valid ○

رابط عمومی برنامه فقط همین دو آرایه، شمار هویت‌های ثبت‌شده و تابع App_TemperatureGetCelsius است. ساختار مدیر، ROM، گذرگاه و توابع تخصیص داخل لایه اتصال برد پنهان مانده‌اند. آرایه‌ها نیز فقط پس از کشف یا خواندن تازه به‌روز می‌شوند، نه در هر دور حلقه اصلی. معماری، معرفی، تعویض و انتقال در فصل ۱۶ تشریح شده است.

۲.۴ اندازه‌گیری انرژی

ماژول قابل حمل ADE7880 روی SPI2 کار می‌کند و وابستگی مستقیم به HAL ندارد. برنامه ولتاژ مؤثر، جریان مؤثر، توان اکتیو، توان ظاهری و ضریب توان هر سه فاز و جریان نول را در snapshot سازگار می‌خواند. ضریب توان از رجیسترهای مستقل APF/BPF/CPF گرفته می‌شود. خطا و timeout به فراخواننده بازمی‌گردند و reset حلقه نامحدود ندارد. شرح کامل اتصال، API، کالیبراسیون و آزمون در فصل ۱۵ آمده است.

۳.۴ رطوبت آنالوگ

کانال فعال ADC پس از هر ۲۰۰۰ نمونه میانگین‌گیری می‌شود. تبدیل فعلی چنین است: ولتاژ مرجع تجربی ۳۳۶.۳ ولت، آفست ۸۵۳.۰ ولت، شیب ۳۰ میلی‌ولت بر درصد و دو ضریب اصلاح ۳.۱ و ۰.۴.۱. خروجی برای جلوگیری از فرمان نامعتبر به بازه صفر تا صد درصد محدود شده است. این ضرایب وابسته به برد و کالیبراسیون‌اند و باید در پرونده کالیبراسیون محصول ثبت شوند.

۴.۴ حسگر رطوبت دیجیتال

ماژول AM2315 در پروژه Keil کامپایل می‌شود و I2C1 نیز راه‌اندازی شده است، اما فراخوانی آن در مسیر اجرایی وجود ندارد. پیاده‌سازی بازبینی‌شده کدهای بازگشت HAL، ساختار پاسخ، علامت دما و CRC16 Modbus را بررسی می‌کند. این ماژول برای سازگاری نگه داشته شده، ولی پیش از فعال‌سازی باید روی سخت‌افزار تست شود.

۵.۴ فیلتر میانگین متحرک

ماژول MAF پیاده‌سازی پنجره ۴۰ نمونه‌ای دارد، اما نه در پروژه Keil افزوده شده و نه در هیچ سورس فعال فراخوانی می‌شود. محاسبه آن در هر به‌روزرسانی کل پنجره را دوباره جمع می‌کند. برای استفاده آینده بهتر است جمع جاری عضو ساختار باشد تا هزینه ثابت شود. در نسخه فعلی حذف نشده، زیرا یک کتابخانه مستقل و قابل استفاده است؛ وضعیت غیرفعال آن باید حفظ شود.

۶.۴ درایورهای HAL

ماژول‌های ADC، DMA، GPIO، I2C، SPI، تایمر، UART، Flash، RCC، توان، Cortex و EXTI در تنظیمات و پروژه Keil حضور دارند. EXTI کاربرد مستقیم ندارد ولی وابستگی زیرساختی HAL است. فایل‌های LL موجود در پوشه درایورها کامپایل نمی‌شوند. آثار آبجکت‌هایی مانند ADE7753، MCP4725 در پوشه خروجی Keil سورس متناظر ندارند و بخشی از محصول فعال نیستند.

فصل ۵

رفتار زمان اجرا و کنترل محیط

۱.۵ راهاندازی

پس از راهاندازی کلاک و پیرامونی‌ها، همه محرک‌های فعال پایین خاموش می‌شوند، گذرگاه‌های دما آزاد می‌شوند، جدول شناسه بارگذاری می‌شود و مدار انرژی به حالت SPI قفل، بازنشانی و اجرا می‌رود. سپس DMA، دریافت تک‌بایتی UART و سه تایمر آغاز می‌شوند. تأخیر پنج‌ثانیه‌ای قدیمی و ارسال I2C با اشاره‌گر نامعتبر حذف شده‌اند.

۲.۵ کنترل رطوبت

در حالت خودکار، پایین‌تر از ۹۸ درصد نقطه تنظیم رطوبت‌ساز روشن و از ۱۰۱ درصد آن به بالا خاموش می‌شود. بین دو مرز وضعیت قبلی حفظ می‌شود. در نسخه قدیمی خرابی حسگر دیجیتال مانع کنترل بود، درحالی‌که حسگر فعال آنالوگ است؛ وابستگی نادرست حذف شده است.

۳.۵ کنترل سرمایش

اگر دمای محیط بیش از نیم درجه بالاتر از نقطه تنظیم باشد، دو خروجی سرمایش روشن می‌شوند. اگر دما نیم درجه یا بیشتر پایین‌تر باشد، خاموش می‌شوند. ناحیه یک‌درجه‌ای میانی هیستریزیس از قطع‌ووصل سریع جلوگیری می‌کند. حفاظت حداقل زمان خاموش یا روشن کمپرسور هنوز وجود ندارد و باید پیش از اتصال مستقیم کمپرسور توسط سخت‌افزار یا نرم‌افزار جداگانه تضمین شود.

۴.۵ کنترل گرمایش

زیر ۹۵ درصد نقطه تنظیم، هیتر پیوسته روشن و از ۱۰۱ درصد به بالا خاموش است. در ناحیه میانی، رابطه تجربی کد قدیمی به یک پنجره صدمرحله‌ای محدود شده است. این امکان روشن ماندن سرمایش و جبران با هیتر را حفظ می‌کند. رابطه تجربی در نقطه تنظیم تقریباً ۱۶ درصد زمان روشن می‌دهد؛ در لبه پایین می‌تواند نزدیک صد درصد شود.

۵.۵ حالت‌ها و ایمنی

فرمان AUTOFF کنترل خودکار را غیرفعال و همه خروجی‌های شناخته‌شده را خاموش می‌کند. فرمان AUTONN آن را فعال می‌کند. فرمان نقطه تنظیم نیز کنترل را مجاز می‌سازد. فرمان‌های دستی مستقیماً خروجی را تغییر می‌دهند، ولی اگر حالت خودکار فعال بماند چرخه بعدی می‌تواند آن را بازنویسی کند؛ نرم‌افزار رایانه باید پیش از کنترل دستی، حالت خودکار را خاموش کند.

هشدار: معنای الکتریکی خروجی‌ها از روی کد «فعال پایین» استنتاج شده است. پیش از نخستین پروگرام نسخه بازآرایی‌شده، حالت خاموش واقعی هر رله و SSR با مولتی‌متر بررسی شود.

فصل ۶

پروتکل سریال رایانه

۱.۶ لایه انتقال

رابط روی ۹۶۰۰ بیت‌برثانیه، هشت بیت داده، یک بیت توقف و بدون توازن است. هر فرمان ۱۲ بایت دارد. اگر فرمان کوتاه‌تر است، فرستنده باید با صفر آن را تا ۱۲ بایت پر کند. هیچ مجموع مقابله با خطا، شماره توالی یا گریز از قاب در نسخه قدیمی تعریف نشده است؛ برای کابل نویزی افزودن نسخه دوم پروتکل توصیه می‌شود.

۲.۶ فرمان‌ها

پیشوند	اثر
S1FF / S1NN	خاموش یا روشن کردن رله ایستگاه اول روی PE2.
S2FF / S2NN	خاموش یا روشن کردن خروجی PE1؛ با فرمان‌های رطوبت هم‌پوشان است.
COLF / COLN	خاموش یا روشن کردن هر دو خروجی سرمایش.
HUMF / HUMN	خاموش یا روشن کردن رطوبت‌ساز.
HETF / HETN	خاموش یا روشن کردن هیتر.
SETP	بایت‌های ۴ و ۵ دما و بایت‌های ۶ و ۷ رطوبت، بدون علامت، کم‌ارزش ابتدا و با مقیاس یک‌دهم.

پیشوند	اثر
TMP	بایت ۳ شماره یک‌مبنای حسگر و بایت‌های ۴ تا ۱۱ شناسه ROM؛ بازه و CRC بررسی و سپس کل جدول ذخیره می‌شود.
AUTOFF و AUTONN	غیرفعال یا فعال کردن کنترل خودکار.
EXIT	بازتاب ۱۲ بایت، خاموش‌سازی خروجی‌ها و بازنشانی پردازنده.

۳.۶ قالب داده ۲۰۹ بایتی

بازه بایت	قالب	معنا
۰ تا ۳	ثابت	سرآیند AA BB CC DD
۴	هشت‌بیتی	مقدار ۲۰۴، طول داده پس از این بایت
۵ تا ۱۲	چهار عدد ۱۶ بیتی	ولتاژ، جریان، توان و ضریب توان ایستگاه اول
۱۳ تا ۲۰	چهار عدد ۱۶ بیتی	همان کمیت‌ها برای ایستگاه دوم
۲۱ تا ۳۶	صفر رزرو	انرژی ایستگاه‌های سوم و چهارم
۳۷ تا ۶۸	شانزده عدد ۱۶ بیتی	دماهای خام ایستگاه اول
۶۹ تا ۱۰۰	شانزده عدد ۱۶ بیتی	دماهای خام ایستگاه دوم
۱۰۱ تا ۱۶۴	صفر رزرو	دمای ایستگاه‌های سوم و چهارم
۱۶۵ تا ۱۶۶	۱۶ بیتی	دمای محیط با واحد یک‌دهم درجه
۱۶۷ تا ۱۶۸	۱۶ بیتی	رطوبت محیط با واحد یک‌دهم درصد
۱۶۹ تا ۱۷۲	دو عدد ۱۶ بیتی	شمارنده استارت دو کانال، ضرب‌شده در ده برای سازگاری
۱۷۳ تا ۱۷۶	صفر رزرو	شمارنده ایستگاه‌های سوم و چهارم
۱۷۷ تا ۲۰۴	دنباله ثابت	مقادیر ۷ تا ۳۴ ده‌دهی؛ نقش کاربردی ناشناخته
۲۰۵ تا ۲۰۸	ثابت	پایان FC FD FE FF

تمام اعداد ۱۶ بیتی به ترتیب کم ارزش ترین بایت و سپس پر ارزش ترین بایت اند. آستانه اعتبار ولتاژ ۱۰۰ تا کمتر از ۳۰۰۰ است؛ بیرون این بازه سه کمیت اصلی همان ایستگاه صفر می شوند. ضریب توان فعلاً همواره صفر است.

فصل ۷

نتیجه ممیزی و تغییرات انجام شده

۱.۷ نقاط قوت نسخه اولیه

تنظیمات اصلی کلاک و پیرامونی‌ها روشن و خروجی ساخت Keil موجود موفق و بدون هشدار بوده است. پروتکل ثابت، ضرایب کالیبراسیون عملیاتی، دو گذرگاه دما، شناسه‌های یکتا و هیستریزیس کنترل، دانش ارزشمند محصول را ثبت کرده‌اند. استفاده از نشانگرهای حفاظت‌شده CubeMX در بخش‌هایی از فایل اصلی نیز پایه مناسبی بوده است.

۲.۷ اشکال‌های بحرانی اصلاح شده

- عملیات طولانی SPI، 1-Wire و UART از متن وقفه خارج شد.
- خواندن مستقیم ثبات UART پس از HAL و فراخوانی دوباره هندلر حذف و دریافت استاندارد وقفه‌ای جایگزین شد.
- منطق کاربردی از فایل وقفه تولیدشده به ماژول مستقل انتقال یافت.
- اختلاف دوره تایمر دوم میان فایل مولد و سورس برطرف شد.
- خواندن از 0x08009000 و نوشتن در 0x08008000 با یک صفحه رزرو شده، نسخه‌دار و دارای CRC جایگزین شد.
- حلقه‌ای که ۶۴ مقدار را از آرایه ۳۲ عضوی می‌خواند حذف شد.
- خطر هم‌پوشانی داده تنظیمات با کد برنامه با کاهش ناحیه لینک برطرف شد.

- بلوک‌های بزرگ کد کامنت شده و متغیرهای بایت به بایت تکراری حذف و ساخت قاب حلقه‌ای شد.
- تقسیم بر صفر در محاسبه پنجره هیترو و خروجی خارج از بازه رطوبت مهار شد.
- حالت خاموش خودکار اکنون واقعاً پایدار و ایمن است.

۳.۷ موارد حفظ شده

قالب ۲۰۹ بایتی، نرخ سریال، نگاشت دو ایستگاه، ترتیب بایت، آدرس‌های پیش فرض، ضرایب کالیبراسیون، منطق فعال پایین، هیستریزیس و رابطه پنجره هیترو حفظ شده‌اند. جابه‌جایی ظاهری شمارنده‌های استارت نیز برای سازگاری قاب قدیمی حفظ و در کد روشن ثبت شده است.

۴.۷ ریسک‌های باز

۱. درایور 1-Wire هنوز زمان تبدیل کافی مطابق بدترین حالت دیتاشیت و CRC داده را تضمین نمی‌کند.
۲. تبدیل‌های طولانی ۳۴ حسگر ترتیبی‌اند و می‌توانند نرخ حلقه را کاهش دهند.
۳. پروتکل ورودی همگام‌سازی، CRC، پاسخ موفقیت و پاسخ خطا ندارد.
۴. حفاظت زمانی کمپرسور، تشخیص اتصال کوتاه حسگر و واچ‌داگ فعال نیست.
۵. نقش چند خروجی GPIO، کانال دوم ADC و دنباله بایت‌های ۱۷۷ تا ۲۰۴ مستند سخت‌افزاری ندارد.
۶. ضرایب انرژی و رطوبت تنها از کد استخراج شده‌اند و شناسنامه کالیبراسیون همراه پروژه نیست.

۵.۷ چرا بعضی فایل‌ها حذف نشدند

کتابخانه‌های AM2315 و MAF سورس مستقل و بالقوه مفیدند و حذفشان برای پروژه‌های بعدی برگشت‌پذیری را کم می‌کند؛ بنابراین با برچسب غیرفعال نگه داشته شدند. در مقابل، خروجی‌های ساخت قدیمی در این مرحله فیزیکی حذف نشدند، چون مسیر فعلی مخزن Git نیست و ممکن است برای مقایسه باینری لازم باشند؛ فایل ignore مانع ورود دوباره آن‌ها به کنترل نسخه خواهد شد.

فصل ۸

ساخت، پروگرام و تولید مجدد

۱.۸ ساخت با Keil

پروژه MDK-ARM/STM32F107.uvprojx را باز کنید، بسته خانواده STM32F1 را در Keil نصب کنید و هدف را بسازید. دو فایل کاربرد و ذخیره‌سازی به گروه کاربر افزوده شده‌اند. اندازه IROM1 برابر 0x3F800 است و نباید به ۲۵۶ کیلوبایت کامل بازگردد. پروگرامر J-Link از SWDIO, SWCLK, GND و ولتاژ مرجع برد استفاده می‌کند.

۲.۸ ساخت با PlatformIO

در ریشه پروژه اجرا کنید:

```
pio run
pio run --target upload
pio device monitor
```

محیط stm32f107vc_jlink از شناسه رسمی برد eval_f107vc، چارچوب STM32Cube و J-Link استفاده می‌کند. فایل لینک اختصاصی آخرین صفحه Flash را رزرو می‌کند. فایل سامانه محلی از ساخت PlatformIO کنار گذاشته می‌شود تا نسخه چارچوب و فایل پروژه هم‌زمان نماد تکراری نسازند.

۳.۸ تولید مجدد با CubeMX

۱. پیش از تولید، از پوشه پروژه نسخه پشتیبان یا یک commit بسازید.
۲. فایل STM32F107.ioc را باز و گزینه حفظ کد کاربر را کنترل کنید.
۳. دوره تایمر دوم باید ۳۵۹۹ و کلاک تایمر ۷۲ مگاهرتز باشد.
۴. کد Keil را تولید کنید و هیچ اخطار حذف فایل را بدون بازبینی نپذیرید.
۵. بررسی کنید فایل‌های app.c و sensor_address_store.c هنوز عضو پروژه Keil باشند.
۶. اندازه Flash لینک Keil را دوباره روی 0x3F800 قرار دهید؛ مولد ممکن است آن را بازنویسی کند.
۷. تفاوت‌ها را بررسی، سپس هر دو ساخت را اجرا کنید.

هشدار: هیچ فایل HEX قدیمی را پس از تغییر سورس به عنوان نتیجه معتبر تلقی نکنید. زمان و خروجی ساخت باید با سورس همان نسخه ثبت شود.

۴.۸ ساخت گزارش

از پوشه docs دستور زیر را دوبار اجرا کنید تا فهرست کامل شود:

```
xelatex -interaction=nonstopmode relay_on_duration_report.tex
```

```
xelatex -interaction=nonstopmode relay_on_duration_report.tex
```

اگر فونت بی‌نازنین نصب باشد خودکار استفاده می‌شود و در غیر این صورت تاهوما جایگزین است. رنگ‌ها، حاشیه‌ها و فونت‌ها تنها در فایل تنظیمات متمرکز قرار دارند.

فصل ۹

تأیید، تحویل و نگهداری آینده

۱.۹ آزمون رومیزی بدون بار قدرت

ابتدا رله‌ها و SSRها را از بار اصلی جدا کنید. سطح خاموش همه خروجی‌ها، کلاک ۷۲ مگاهرتز، نرخ UART و انتخاب تراشه SPI را با ابزار اندازه‌گیری کنترل کنید. سپس یک حسگر روی هر گذرگاه وصل و شناسه، دمای خام و دمای محیط را با دماسنج مرجع مقایسه کنید.

۲.۹ آزمون پروتکل

برای هر فرمان یک قاب دقیق ۱۲ بیتی بفرستید و اثر خروجی را ثبت کنید. قاب ۲۰۹ بیتی را از نظر سرآیند، انتها، طول، ترتیب کم‌ارزش‌ترین بایت و جایگاه هر حسگر بررسی کنید. سناریوهای قطع کابل، بایت گمشده، فرمان نامعتبر و توقف نرم‌افزار رایانه نیز باید آزموده شوند.

۳.۹ آزمون کنترل

با بار شبیه‌ساز، نقاط پایین، داخل و بالای هیستریزیس دما و رطوبت را اعمال کنید. گذار خودکار به دستی و برعکس، بازنشانی، قطع حسگر و بازیابی برق را ثبت کنید. شمارنده استارت باید تنها روی لبه خاموش به روشن افزایش یابد. سپس حداقل زمان مجاز کمپرسور طبق سازنده به آزمون اضافه شود.

۴.۹ معیار پذیرش نسخه

- ساخت Keil و PlatformIO بدون خطا انجام شود.
- اندازه کد از مرز 0x0803F800 عبور نکند.
- تولید مجدد CubeMX هیچ کد کاربردی را حذف نکند.
- خاموشی ایمن هنگام راه اندازی، خطا و AUTOFF تأیید شود.
- همه نگاشت‌های شماره گذاری شده پس از قطع برق سالم بازیابی و CRC رکورد و ROM آن‌ها تأیید شود.
- نرم افزار رایانه قاب جدید را بدون تغییر تفسیر کند.

۵.۹ اولویت‌های توسعه بعدی

بازنویسی قابل حمل 1-Wire، تبدیل هم‌زمان، کنترل CRC و شماره گذاری پایدار انجام شده است. اولویت بعدی آزمون سخت‌افزاری این مسیر با تعداد و کابل واقعی، سپس پروتکل نسخه‌دار با CRC و پاسخ، ماشین حالت حفاظت کمپرسور، واچ‌داگ، گزارش خطای حسگر و تست واحد میزبان است. پس از دریافت شماتیک می‌توان نام و وضعیت پایه‌های ناشناخته را قطعی و ماژول‌های بلااستفاده را حذف یا فعال کرد.

۶.۹ قاعده نگهداری سند

هر تغییر رفتاری باید هم‌زمان در فصل مربوط، فایل وضعیت ریشه، شماره نسخه پروتکل و فهرست آزمون ثبت شود. نتیجه آزمون سخت‌افزاری نباید از روی موفقیت کامپایل استنتاج شود. فایل وضعیت پروژه نقطه شروع هر عامل انسانی یا هوش مصنوعی برای ادامه کار است.

فصل ۱۰

مرجع کامل توابع لایه کاربرد

این فصل مرجع نسخه فعال لایه کاربرد در فایل‌های Application/Inc/app.h و Application/Src/app.c است. فایل مولد Core/Src/main.c فقط پس از آماده‌شدن پایه‌ها و SPI2 تابع راه‌اندازی را صدا می‌زند و سپس تابع پردازش را در حلقه اصلی اجرا می‌کند. منطق ADE7880 و چراغ‌های عیب‌یابی بیرون از فایل مولد باقی می‌ماند.

۱.۱۰ حالت عمومی قابل مشاهده در دیباگر

برای دما، رابط معمول سه نماد ساده دارد:

○ `g_temperature_c[N-1]` دمای سنسور شماره `N`؛

○ `g_temperature_valid[N-1]` اعتبار همان مقدار؛

○ `g_temperature_sensor_count` تعداد هویت‌های ثبت‌شده.

کشف و تخصیص نخستین شماره خالی خودکار است و برنامه عادی تابعی را از دیباگر فراخوانی نمی‌کند. تابع `App_TemperatureGetCelsius` فقط هنگام وجود نمونه معتبر `true` برمی‌گرداند. این تنها تابع عمومی دما در لایه کاربرد است؛ API‌های کشف، تخصیص و پاک‌کردن و همچنین ساختار مدیر از هدر ساده حذف شده‌اند تا کد مصرف‌کننده به جزئیات ROM، گذرگاه و ماشین حالت وابسته نشود.

متغیر سراسری `g_app_electrical` از نوع `App_ElectricalState` است و برای پنجره Watch طراحی شده است. اندیس صفر، یک و دو همیشه به ترتیب فازهای A/B/C هستند. مهم‌ترین اعضا عبارت‌اند از:

عضو	معنی و شرط استفاده
online	آخرین راه اندازی و نمونه کامل ارتباطی موفق بوده است.
last_status	نتیجه دقیق آخرین عملیات کتابخانه.
successful_samples	شمار نمونه های کامل و موفق.
communication_errors	شمار خطاهای خواندن پس از برقراری ارتباط.
raw.phase[0..2]	شمارهای مستقیم رجیستر برای ولتاژ، جریان، W، VA و PF هر فاز.
voltage_v[0..2]	ولتاژ RMS برحسب ولت، فقط وقتی valid متناظر درست است.
current_a[0..2]	جریان RMS برحسب آمپر، فقط وقتی valid متناظر درست است.
active_power_w[0..2]	توان اکتیو علامت دار برحسب وات.
apparent_power_va[0..2]	توان ظاهری برحسب ولت آمپر.
power_factor[0..2]	ضریب توان علامت دار از منفی یک تا مثبت یک.
power_factor_abs[0..2]	بزرگی ضریب توان از صفر تا یک.
neutral_current_a	جریان RMS نول برحسب آمپر.
..._source	منشأ ضریب: بدون ضریب، پیش فرض آغاز کار یا ضریب کاربر.

هیچ مقدار مهندسی بدون بررسی پرچم valid متناظر استفاده نشود. هنگام تغییر ضریب یا جهت CT مقدار قبلی و خروجی نمایشی همان لحظه صفر و valid نادرست می شوند؛ مقدار تازه با نخستین نمونه موفق بعدی منتشر می شود.

۲.۱۰ توابع اصلی چرخه اجرا

۱.۲.۱۰ تابع App_Init

ورودی، نشانی دسته SPI است که پیش‌تر توسط MX_SPI2_Init راه‌اندازی شده است. تابع مراحل زیر را انجام می‌دهد:

۱. مازول چراغ عیب‌یابی و تمام حالت عمومی را مقداردهی اولیه می‌کند؛
۲. واحدهای نمایشی پیش‌فرض و جهت عادی سه CT را قرار می‌دهد؛
۳. جدول کامل ضرایب آغازکار A/B/C و جریان نول را بارگذاری می‌کند؛
۴. callback‌های مستقل از میکرو را به توابع HAL متصل می‌کند؛
۵. رابط SPI را انتخاب، تراشه را بازنشانی و پردازشگر اندازه‌گیری را اجرا می‌کند؛
۶. نتیجه را در حالت عمومی و الگوی چراغ ثبت می‌کند.

اشاره‌گر تهی پذیرفته نمی‌شود، ولی برنامه در راه‌اندازی قفل نمی‌کند؛ وضعیت خطا ثبت و حلقه اصلی بعداً تلاش مجدد انجام می‌دهد.

۲.۲.۱۰ تابع App_Process

این تابع ورودی و خروجی ندارد و باید پیوسته در (1)while فراخوانی شود. در هر اجرا ابتدا چراغ غیرمسدودکننده را سرویس می‌کند. اگر موعد نرسیده باشد بلافاصله بازمی‌گردد. در حالت آنلاین هر یک ثانیه یک نمونه کامل سه‌فاز و نول می‌خواند؛ در شکست، حالت آفلاین و الگوی خطا فعال می‌شوند. در حالت آفلاین هر دو ثانیه راه‌اندازی دوباره آزمایش می‌شود.

داده ابتدا در متغیر محلی خوانده می‌شود. فقط اگر همه رجیسترها موفق باشند، raw و تمام مقدارهای مهندسی با هم منتشر می‌شوند؛ بنابراین ساختار عمومی هرگز مخلوطی از فاز تازه و قدیمی ندارد.

۳.۱۰ ضرایب پیش‌فرض و بازنشانی

۱.۳.۱۰ تابع App_ResetCalibrationToDefaults

جدول ثابت default_calibration را برای ولتاژ، جریان، توان اکتیو و توان ظاهری هر سه فاز و جریان نول کپی می‌کند. همه مقدارهای قبلی صفر و پرچم‌های valid نادرست می‌شوند. منبع همه خانه‌ها مقدار APP_CALIBRATION_DEFAULT می‌گیرد. سپس خروجی‌های نمایش نیز تازه می‌شوند.

این تابع در App_Init خودکار اجرا می‌شود و در زمان اجرا نیز برای کنار گذاشتن همه ضرایب کاربر قابل فراخوانی است. ضرایب پیش‌فرض فاز C و نول موقت‌اند و باید با مرجع واقعی تأیید شوند.

۴.۱۰ نصب و محاسبه کالیبراسیون فاز

۱.۴.۱۰ تابع App_SetVoltageCalibration

یک رابط کوتاه سازگار برای نصب scale/offset ولتاژ یک فاز است و درون خود تابع عمومی نصب فاز را فراخوانی می‌کند.

۲.۴.۱۰ تابع App_SetPhaseCalibration

سه ورودی دارد: نوع کمیت، فاز و نشانی ضریب خطی. کمیت مجاز یکی از ولتاژ RMS، جریان RMS، توان اکتیو یا توان ظاهری است. تابع فقط خانه انتخاب‌شده را جایگزین می‌کند، source همان خانه را کاربر قرار می‌دهد و مقدار قبلی و نمایش وابسته را تا نمونه بعدی پاک می‌کند. آرگومان تهی، فاز نامعتبر یا کمیت خارج از enum با ADE7880_STATUS_INVALID_ARGUMENT رد می‌شود.

۳.۴.۱۰ تابع App_CalibratePhaseMultiPoint

دو یا چند زوج میانگین raw/reference را به تابع قابل‌حمل کمترین مربعات می‌دهد، یک خط engineering $offset + scale * raw$ می‌سازد و آن را برای کمیت و فاز انتخابی نصب می‌کند. سه تا پنج نقطه دور از هم توصیه می‌شود. دو خروجی اختیاری، ضریب دقیق نصب‌شده و بیشترین خطای باقیمانده نقاط را برمی‌گردانند. اگر جهت نرم‌افزاری CT برای توان اکتیو معکوس باشد، تابع علامت ضرایب داخلی را طوری هماهنگ می‌کند که reference همچنان خروجی نهایی مورد انتظار کاربر باشد.

۵.۱۰ کالیبراسیون جریان نول

۱.۵.۱۰ تابع App_SetNeutralCurrentCalibration

یک جفت ضریب را برای رجیستر NIRMS نصب می‌کند. مقدار قبلی صفر، valid نادرست و source برابر کاربر می‌شود. مقدار جدید پس از نمونه موفق بعدی منتشر خواهد شد.

۲.۵.۱۰ تابع `App_CalibrateNeutralCurrentMultiPoint`

همان برازش چندنقطه‌ای را با reference برحسب آمپر انجام می‌دهد و نتیجه را از مسیر تابع نصب نول فعال می‌کند. ضریب و بیشترین خطای باقیمانده هر دو خروجی اختیاری هستند.

۶.۱۰ جهت ترانس جریان و واحد نمایش

۱.۶.۱۰ تابع `App_SetCurrentPolarity`

فاز و یکی از دو مقدار NORMAL/REVERSED را می‌گیرد. حالت معکوس فقط علامت توان اکتیو و PF علامت‌دار را اصلاح می‌کند؛ جریان RMS، توان ظاهری و بزرگی PF تغییر نمی‌کنند. پس از تغییر، مقادیر علامت‌دار قبلی و نمایش توان همان لحظه پاک و تا نمونه بعدی نامعتبر می‌شوند.

تشخیص خودکار جهت انجام نمی‌شود، زیرا توان منفی ممکن است صادرات واقعی باشد. جهت باید یک بار هنگام راه‌اندازی فیزیکی تعیین شود.

۲.۶.۱۰ تابع `App_SetDisplayUnits`

دو ضریب مثبت برای نمایش جریان و توان می‌گیرد. پیش‌فرض‌ها ۱۰۰ واحد بر آمپر و ۱۰ واحد بر وات/ولت‌آمپر هستند؛ بنابراین 0.17 A به شکل ۱۷ و 23.1 W به شکل ۲۳۱ دیده می‌شوند. این تابع فقط فیلدهای نمایش را تغییر می‌دهد و هیچ اثری روی SI، raw یا کالیبراسیون ندارد.

۷.۱۰ توابع داخلی مهم

تابع	مسئولیت
<code>update_display_values</code>	ساخت خروجی نمایشی فقط از مقدار SI معتبر.
<code>publish_linear_value</code>	اعمال یک ضریب یا انتشار صفر و valid نادرست.
<code>update_engineering_values</code>	تبدیل متقارن A/B/C، اصلاح جهت و تبدیل Q1.15 مربوط به PF.

تابع	مسئولیت
initialize_ade	اتصال رابط انتقال، اجرای آغاز کار و انتخاب زمان نمونه یا تلاش مجدد.
phase_calibration_slot	نگاشت enum کمیت به خانه درست ساختار ضرایب.
invalidate_phase_value	پاک کردن مقدار قبلی و ثبت منبع ضریب جدید.

چهار رابط محلی نیز نتیجه HAL را به وضعیت کتابخانه تبدیل و عملیات ارسال، دریافت، انتخاب تراشه و تأخیر را فراهم می کنند. این وابستگی ها فقط در لایه کاربرد هستند؛ کتابخانه ADE7880 هیچ هدر STM32 را شامل نمی کند.

۸.۱۰ الگوی استفاده تک فاز و سه فاز

برای محصول تک فاز فقط اندیس ADE7880_PHASE_A مصرف می شود. برای دو کانال، A و B مستقل اند. محصول سه فاز A/B/C را می خواند. نصب کالیبراسیون یک خانه هیچ ضریب دیگری را عوض نمی کند. در پروژهای با چند تراشه نیز برای هر IC یک ADE7880_Device و رابط انتقال مستقل ساخته می شود؛ نمونه سراسری داخل کتابخانه وجود ندارد.

۹.۱۰ چک لیست استفاده صحیح

۱. ابتدا online و last_status کنترل شوند.
۲. مقدار raw و مقدار مهندسی متناظر هم زمان دیده شوند.
۳. هیچ مقدار مهندسی بدون valid متناظر مصرف نشود.
۴. source پیش فرض به معنی ضریب موقت آغاز کار است؛ برای دقت نهایی هر کانال جدا کالیبره شود.
۵. ضریب بازگشتی تأیید شده در کد یا حافظه غیرفرار ذخیره شود، زیرا ضریب زمان اجرا با reset از بین می رود.
۶. ثانویه CT هنگام عبور جریان اولیه هرگز باز نشود.

فصل ۱۱

مرجع کامل توابع درایورها

۱.۱۱ ذخیره شناسه حسگر

۱.۱.۱۱ تابع `crc8_maxim`

ورودی آن اشاره گر داده و طول است و CRC هشت بیتی بازتاب یافته با چند جمله ای 0x8C را برمی گرداند. تابع فقط داخلی است و برای هفت بایت نخست شناسه یک سیمه استفاده می شود.

۲.۱.۱۱ تابع `crc32_record`

روی تمام بایت های رکورد به جز فیلد CRC نهایی اجرا می شود. مقدار آغازین همه یک، چند جمله ای بازتاب یافته 0xEDB88320 و خروجی معکوس شده است. هدف تشخیص رکورد ناقص، فرسوده یا متعلق به نسخه دیگر است.

۳.۱.۱۱ تابع `SensorAddressStore_IsValidRom`

یک شناسه هشت بیتی می گیرد. اشاره گر تهی، کد خانواده غیر از ۲۸ یا CRC نامنطبق نتیجه نادرست می دهد. این تابع هم برای فرمان جدید رایانه و هم اعتبارسنجی کل جدول استفاده می شود.

۴.۱.۱۱ تابع `table_is_valid`

تمام ۳۲ ردیف جدول را به تابع اعتبارسنجی شناسه می‌دهد و با نخستین خطا بازمی‌گردد. سیاست فعلی جدول ناقص را نمی‌پذیرد؛ بنابراین تعویض یک حسگر تنها وقتی ذخیره می‌شود که همه ردیف‌های دیگر نیز معتبر باشند.

۵.۱.۱۱ تابع `SensorAddressStore_Load`

رکورد صفحه پایانی Flash را بدون کپی اولیه بررسی می‌کند. امضا، نسخه، تعداد، CRC رکورد و CRC تک‌تک شناسه‌ها باید درست باشند. در موفقیت، جدول به خروجی کپی می‌شود. در شکست، جدول خام قدیمی بررسی می‌شود و در صورت اعتبار، علاوه بر کپی خروجی به قالب جدید ذخیره می‌شود. اگر هر دو نامعتبر باشند نتیجه نادرست است و لایه کاربرد از پیش‌فرض‌ها استفاده می‌کند.

۶.۱.۱۱ تابع `SensorAddressStore_Save`

ابتدا کل جدول ورودی را اعتبارسنجی و یک رکورد محلی می‌سازد. سپس Flash را باز، دقیقاً یک صفحه را پاک و رکورد را کلمه‌به‌کلمه می‌نویسد و در پایان Flash را قفل می‌کند. هر خطای HAL نتیجه نادرست می‌دهد. چون پاک کردن صفحه برگشت‌ناپذیر و نوشتن مسدودکننده است، تابع فقط در حلقه اصلی فراخوانی می‌شود.

۲.۱۱ درایور ADE7880

هشدار: این بخش رفتار نسخه قدیمی درایور را برای سابقه بازآرایی ثبت می‌کند. مرجع API فعال و اصلاح‌شده، فصل ۱۵ و فایل `Libraries/ADE7880/Inc/ade7880.h` است.

۱.۲.۱۱ تابع `ADE7880_ReadRegister`

ورودی‌ها نمونه درایور، نشانی ۱۶ بیتی رجیستر، تعداد بایت و مهلت HAL هستند. بافر چهاربایتی داخلی صفر، فرمان خواندن و نشانی ارسال، سپس یک تا چهار بایت دریافت می‌شود. انتخاب تراشه پیرامون کل تراکنش پایین و بالا می‌رود. داده در TEMP نمونه درایور قرار می‌گیرد و وضعیت خطای HAL هنوز به فراخواننده بازگردانده نمی‌شود.

۲.۲.۱۱ تابع ADE7880_WriteRegister

داده ۳۲بیتی را متناسب با طول یک، دو یا چهار بایت با ترتیب پرارزش‌ترین بایت آماده می‌کند، فرمان نوشتن و نشانی را پیشوند می‌دهد و تراکنش SPI را زیر انتخاب تراشه انجام می‌دهد. طول‌های دیگر نباید ارسال شوند و در نسخه آینده بهتر است نوع نتیجه برای خطا افزوده شود.

۳.۲.۱۱ تابع ADE7880_ConfigSPI

دسته SPI و پایه انتخاب تراشه را در نمونه درایور ذخیره می‌کند. سه بار انتخاب تراشه را تغییر می‌دهد تا رابط سریال مدار روی SPI قرار گیرد، سپس رجیستر پیکربندی دوم را می‌نویسد تا انتخاب قفل شود. پس از بازنشانی سخت‌افزاری باید دوباره اجرا شود.

۴.۲.۱۱ تابع ADE7880_Run

رجیستر اجرای پردازشگر داخلی مدار را با مقدار یک می‌نویسد. ورودی فقط نمونه درایور است و پس از ارسال یک میلی‌ثانیه صبر می‌کند.

۵.۲.۱۱ تابع ADE7880_ReadVRMS

سه رجیستر ولتاژ مؤثر فازهای A تا C را به ترتیب می‌خواند. هر مقدار ۲۴بیتی از سه بایت پایانی بافر ساخته و در ساختار فاز ذخیره می‌شود. تبدیل به ولت در این تابع انجام نمی‌شود.

۶.۲.۱۱ تابع ADE7880_ReadIRMS

مانند تابع ولتاژ است، با این تفاوت که سه جریان فاز و جریان نول را تازه می‌کند. خروجی‌ها هنوز شمار خام رجیستر هستند.

۷.۲.۱۱ تابع ADE7880_ReadPowerz

توان حقیقی و توان ظاهری هر سه فاز را از شش رجیستر ۳۲بیتی می‌خواند. داده در اعضای WATT و VA قرار می‌گیرد. علامت توان حقیقی باید مطابق قالب دیتاشیت تفسیر شود؛ ساختار از نوع علامت‌دار استفاده می‌کند.

۸.۲.۱۱ تابع ADE7880_ReadFPower

چهار رجیستر توان حقیقی، راکتیو، ظاهری و ضریب توان بنیادی را می‌خواند و براساس ورودی انتخاب فاز در ساختار A، B یا C می‌گذارد. این رجیسترها اشتراکی‌اند؛ بنابراین فراخواننده باید پیش‌تر بیت انتخاب فاز رجیستر پیکربندی را تنظیم کرده باشد. برنامه فعلی این تابع را استفاده نمی‌کند.

۹.۲.۱۱ تابع ADE7880_Reset

بیت بازنشانی نرم‌افزاری را در رجیستر پیکربندی می‌نویسد و همان رجیستر را تا صفرشدن بیت بررسی می‌کند. حلقه مهلت کلی ندارد؛ قطع ارتباط SPI می‌تواند راه‌اندازی را متوقف کند. افزودن شمارنده تلاش یکی از اصلاحات ضروری آینده است.

۳.۱۱ پشته DS18B20 و OneWire

۱.۳.۱۱ توابع پایه OneWire

تابع OneWire_Init اتصال سخت‌افزاری مبتنی بر callback را اعتبارسنجی و خط را آزاد می‌کند. توابع OneWire_Reset، OneWire_WriteBit و OneWire_ReadBit زمان‌بندی استاندارد را اجرا می‌کنند. نمونه‌های بایتی، هشت بیت را کم‌ارزش‌ترین بیت در ابتدا جابه‌جا می‌کنند. OneWire_Crc8 چندجمله‌ای دالاس/مکسیم را برای ROM و حافظه موقت محاسبه می‌کند.

۲.۳.۱۱ جست‌وجوی شناسه

OneWire_SearchFirst حالت جست‌وجو را پاک و نخستین ROM معتبر را برمی‌گرداند. OneWire_SearchNext اختلاف بعدی در درخت بیت‌ها را دنبال می‌کند. ساختار OneWire_SearchState متعلق به فراخواننده است؛ بنابراین چند گذرگاه یا چند نمونه کتابخانه متغیر سراسری مشترک ندارند.

۳.۳.۱۱ فرمان‌های DS18B20

DS18B20_StartConversion برای یک ROM مشخص یا با ورودی تهی برای همه حسگرهای گذرگاه

فرمان تبدیل می‌فرستد. DS18B20_ReadScratchpad نه بایت را می‌خواند و پیش از موفقیت CRC را کنترل می‌کند؛ وقتی دقیقاً یک حسگر روی گذرگاه باشد، ROM این تابع و تابع blocking می‌تواند تهی باشد. DS18B20_DecodeRaw علامت دما را حفظ و DS18B20_RawToCelsius واحد یک‌شانزدهم درجه را به سلسیوس تبدیل می‌کند. مسیر blocking برای پروژه کوچک مناسب است و runtime محصول از مدیر غیرمسدودکننده استفاده می‌کند.

۴.۳.۱۱ مدیر چندگذرگاهی

DS18B20_ManagerInit پیکربندی، زمان و callbackهای ذخیره را دریافت و نگاشت معتبر را بارگذاری می‌کند. ثبت شناسه تازه و تعویض بدون ابهام رفتار ثابت آن است. DS18B20_ManagerDiscover همه گذرگاه‌ها را جست‌وجو و حضور شماره‌های ذخیره‌شده را تازه می‌کند.

APIهای تخصیص و پاک‌کردن برای ابزار تعمیراتی باقی مانده‌اند. DS18B20_ManagerProcess تبدیل سراسری، انتظار موعود و حداکثر یک خواندن آدرس‌دار را پیش می‌برد و فقط هنگام تغییر اطلاعات عمومی true برمی‌گرداند. جزئیات نگاشت و خطاها در فصل ۱۶ آمده است.

۴.۱۱ درایور AM2315

۱.۴.۱۱ تابع am2315_crc16

تابع داخلی، CRC شانزده‌بیتی بازتاب‌یافته مودباس را با مقدار آغازین 0xFFFF و چندجمله‌ای 0xA001 محاسبه می‌کند. ورودی داده و طول و خروجی CRC است.

۲.۴.۱۱ تابع TEMP_HUM

چهار ورودی آن دسته I2C، بافر فرمان سه‌بایتی، بافر پاسخ هشت‌بایتی و آرایه دو عدد اعشاری خروجی است. ابتدا حسگر را با مرحله نشانی بیدار، سپس فرمان خواندن چهار رجیستر را ارسال و پاسخ را دریافت می‌کند. کد تابع، طول، CRC، علامت دما و وضعیت HAL بررسی می‌شوند. در موفقیت رطوبت و دما نوشته و رطوبت بازگردانده می‌شود؛ در شکست هر دو خروجی صفر و پرچم خطا یک می‌شود. این مازول در مسیر فعال فعلی فراخوانی نمی‌شود.

۵.۱۱ فیلتر MAF

۱.۵.۱۱ تابع MAF_Init

اشاره‌گر یک نمونه فیلتر را می‌گیرد، معکوس طول پنجره را محاسبه، میانگین و همه ۴۰ خانه را صفر و اشاره‌گر نوشتن را روی خانه نخست قرار می‌دهد. باید پیش از به‌روزرسانی فراخوانی شود.

۲.۵.۱۱ تابع MAF_Update

نمونه جدید را در خانه جاری می‌نویسد و اشاره‌گر را حلقوی جلو می‌برد. سپس همه ۴۰ خانه را جمع و میانگین را ذخیره و برمی‌گرداند. ورودی دوم داده جدید است. هزینه هر فراخوانی خطی است و نمونه‌های صفر اولیه در شروع روی نتیجه اثر دارند.

۳.۵.۱۱ تابع Average

اشاره‌گر آرایه و تعداد اعضا را می‌گیرد و میانگین حسابی برمی‌گرداند. اندازه صفر کنترل نشده و باعث تقسیم بر صفر می‌شود؛ فراخواننده موظف است اندازه مثبت بدهد. این تابع و کل ماژول در برنامه فعلی استفاده نمی‌شوند.

فصل ۱۲

شرح جزئی بازآرایی و مرز تغییرات

۱.۱۲ وضعیت پیش از بازآرایی

فایل وقفه علاوه بر هندلرهای مولد، مالک تقریباً همه حالت‌های برنامه، ۲۰۹ متغیر پراکنده قاب، خواندن ۳۴ حسگر، تراکنش‌های انرژی، محاسبات اعشاری، ارسال مسدودکننده و تفسیر فرمان بود. فایل اصلی نیز آدرس‌ها، کنترل خروجی و callback مبدل آنالوگ را نگه می‌داشت. این پراکندگی وابستگی دوطرفه با تعداد زیادی extern ساخته و تولید مجدد CubeMX را پرخطر کرده بود.

۲.۱۲ انتقال مالکیت

تمام داده‌های محصول به متغیرهای ایستای app.c منتقل شدند. فایل اصلی اکنون فقط دسته‌های HAL را در یک ساختار می‌گذارد. فایل وقفه فقط هندلر تولیدشده را صدا می‌زند و callback‌های HAL در فایل کاربرد قرار دارند. در نتیجه جریان وابستگی یک‌طرفه است: فایل مولد، رابط عمومی کاربرد را می‌شناسد؛ کاربرد از دسته‌های تزریق‌شده استفاده می‌کند؛ درایورها از سیاست محصول آگاه نیستند.

۳.۱۲ تبدیل وقفه به کار معوق

پیش‌تر تایمر اول داخل وقفه ۲۰۹ بایت ارسال می‌کرد؛ تایمر دوم همه تراکنش‌های SPI و محاسبه رطوبت را انجام می‌داد؛ تایمر سوم ۳۴ تراکنش دما داشت. اکنون callback تایمر فقط یک پرچم منطقی می‌نویسد و عملیات در حلقه اصلی اجرا می‌شود. این تغییر زمان انسداد وقفه را به چند دستور کاهش می‌دهد و از گرسنگی UART و تداخل زمان‌بندی جلوگیری می‌کند.

۴.۱۲ فشرده‌سازی قاب مخابراتی

متغیرهای جداگانه بایت پایین و بالا حذف شدند. کمیت‌ها در ساختار ۱۶بیتی نگه داشته و هنگام ساخت قاب با تابع واحد سریال می‌شوند. آرایه ابتدا صفر است؛ بنابراین ناحیه‌های رزرو بدون ده‌ها انتساب صریح صفر باقی می‌مانند. حلقه دما جای ۶۴ انتساب تکراری را گرفته است. جایگاه هیچ فیلد سازگار تغییر نکرده است.

۵.۱۲ بازطراحی دریافت سریال

کد قبلی پس از اجرای هندلر HAL، ثبات داده را مستقیم می‌خواند و همان هندلر را دوباره صدا می‌زد. این ترتیب می‌توانست پرچم دریافت را پیش از خواندن پاک یا یک بایت را دوبار رسیدگی کند. نسخه جدید دریافت رسمی تک‌بایتی HAL را شروع می‌کند، callback فقط بافر را پر می‌کند و تفسیر دوازده‌بایت در حلقه اصلی انجام می‌شود. نوشتن Flash دیگر هرگز داخل ISR نیست.

۶.۱۲ بازطراحی حافظه شناسه‌ها

نسخه قدیمی از 0x08009000 می‌خواند، ولی هنگام فرمان تغییر، صفحه 0x08008000 را پاک می‌کرد. سپس ۶۴ مقدار هشت‌بایتی را از آرایه‌ای با ۳۲ عضو می‌نوشت. هر دو نشانی نزدیک انتهای کد ۳۲ کیلوبایتی قدیمی بودند و رشد برنامه می‌توانست داده یا کد را تخریب کند.

نسخه جدید آخرین صفحه واقعی قطعه را رزرو کرده است. لینک Keil و PlatformIO پیش از آن پایان می‌یابد. فقط ۳۲ شناسه نوشته می‌شوند و دو سطح CRC همراه با نسخه قالب وجود دارد. مهاجرت فقط جدول قدیمی کاملاً معتبر را می‌پذیرد. اگر برق میان پاک‌کردن و نوشتن قطع شود، رکورد نامعتبر تشخیص داده و پیش‌فرض استفاده می‌شود؛ نگهداری دو نسخه افزونه‌ای برای تحمل کامل قطع برق هنوز پیاده نشده است.

۷.۱۲ اصلاح رفتار کنترل

رابطه‌ها و ضرایب اصلی حفظ شدند، ولی تقسیم بر نقطه تنظیم صفر، رطوبت خارج از بازه و محاسبه پنجره بزرگ‌تر از صد مهار شدند. حالت خاموش خودکار دیگر در دور بعدی حلقه ناخواسته دوباره فعال نمی‌شود. فرمان دستی همچنان ممکن است در حالت خودکار توسط کنترل بعدی بازنویسی شود؛ این رفتار سازگار و در فصل پروتکل ثبت شده است.

۸.۱۲ مرزهای حفاظت شده CubeMX

در `main.c` فقط افزودن سرآیند کاربرد، ساخت ساختار دسته‌ها، فراخوانی راه‌اندازی و فراخوانی حلقه داخل نشانگرهای کاربر قرار دارد. فایل‌های `app.c`، `sensor\_address\_store.c` و درایورها خارج از مالکیت مولدند. با این حال، فایل پروژه Keil و اندازه حافظه ممکن است توسط مولد بازنویسی شوند و پس از هر تولید باید مطابق چک‌لیست فصل ساخت کنترل شوند.

۹.۱۲ مواردی که عمداً تغییر نکردند

نرخ سریال، طول فرمان، قاب ۲۰۹ بیتی، ترتیب کم‌بایت، شناسه‌های پیش‌فرض، نگاشت فازهای A و B، ضرایب کالیبراسیون، دو خروجی هم‌زمان سرمایش و منطق امکان کار هم‌زمان سرمایش و هیتر حفظ شده‌اند. بدون نرم‌افزار رایانه و شماتیک، تغییر این قراردادها ریسک ناسازگاری داشت.

۱۰.۱۲ موارد باقیمانده برای مرحله بعد

درایور یک‌سیمه باید به ماشین حالت غیرمسدودکننده با پالس زمان‌سنج، تبدیل هم‌زمان، تشخیص حضور و CRC حافظه موقت تبدیل شود. رابط انرژی باید نتیجه HAL و مهلت بازنشانی را برگرداند. پروتکل به همگام‌سازی، CRC، پاسخ و شماره نسخه نیاز دارد. حفاظت حداقل زمان کمپرسور، واچ‌داگ، ثبت خطا و آزمون واحد نیز هنوز انجام نشده‌اند.

فصل ۱۳

مرجع توابع تولیدشده و راهاندازی سختافزار

این فصل بخش‌هایی را شرح می‌دهد که عمدتاً توسط STM32CubeMX تولید می‌شوند. توضیحات برای فهم رفتارند؛ کامنت دستی خارج از ناحیه‌های کاربر نباید به این فایل‌ها افزوده شود.

۱.۱۳ فایل اصلی

۱.۱.۱۳ تابع main

نقطه ورود پس از کد راهانداز پردازنده است. ابتدا HAL، کلاک سامانه و به‌ترتیب پایه‌ها، DMA، SPI، UART، سه تایمر، I2C و ADC را راهاندازی می‌کند. سپس ساختار دسته‌های کاربرد ساخته و App_Init فراخوانی می‌شود. حلقه بی‌نهایت فقط App_Process را اجرا می‌کند. مقدار بازگشت نظری عدد صحیح است، ولی حلقه هرگز پایان نمی‌یابد.

۲.۱.۱۳ تابع SystemClock_Config

نوسان‌ساز خارجی هشت مگاهرتز، تقسیم ورودی یک و ضریب PLL برابر نه را انتخاب می‌کند. کلاک پردازنده و گذرگاه پرسرعت ۷۲ مگاهرتز، گذرگاه اول ۳۶ مگاهرتز و گذرگاه دوم ۷۲ مگاهرتز می‌شوند. کلاک ADC از تقسیم شش و برابر ۱۲ مگاهرتز است. هر خطای HAL به Error_Handler می‌رود.

۳.۱.۱۳ تابع MX_ADC1_Init

مبدل نخست را در حالت اسکن و تبدیل پیوسته با دو رتبه تنظیم می کند. کانال ۵ رتبه اول و کانال ۴ رتبه دوم اند و هر دو زمان نمونه برداری ۵.۲۳۹ سیکل دارند. هم ترازوی داده راست و شروع نرم افزاری است. DMA در تابع پشتیبانی پیرامونی به دسته متصل می شود.

۴.۱.۱۳ تابع MX_I2C1_Init

رابط نخست را با سرعت صد کیلوهرتز، آدرس دهی هفت بیتی، بدون آدرس دوم، فراخوان عمومی یا کشش غیر فعال راه اندازی می کند. پایه ها در تابع پشتیبانی به PB8 و PB9 بازنگاشت می شوند.

۵.۱.۱۳ تابع MX_SPI2_Init

رابط دوم را ارباب تمام دوطرفه، هشت بیتی، پرارزش ترین بیت ابتدا، قطبیت بالا و نمونه برداری لبه دوم می کند. انتخاب تراشه نرم افزاری و تقسیم کلاک ۶۴ است؛ با کلاک گذرگاه ۳۶ مگاهرتز نرخ تقریبی ۵.۵۶۲ کیلوبیت بر ثانیه حاصل می شود.

۶.۱.۱۳ توابع MX_TIM1_Init، MX_TIM2_Init و MX_TIM3_Init

هر سه تایمر کلاک داخلی و شمارش افزایشی دارند. پیش تقسیم گر ۹۹۹۹ کلاک ۷۲ مگاهرتز را به ۷۲۰۰ هرتز می رساند. دوره ۷۱۹۹ در تایمرهای یک و سه رویداد یک ثانیه ای و دوره ۳۵۹۹ در تایمر دو رویداد نیم ثانیه ای می سازد. پیش بارگذاری خودکار غیر فعال است.

۷.۱.۱۳ تابع MX_USART2_UART_Init

رابط دوم را با نرخ ۹۶۰۰، هشت بیت داده، یک توقف، بدون توازن و کنترل جریان، ارسال و دریافت هم زمان و نمونه برداری شانزده برابر تنظیم می کند.

۸.۱.۱۳ تابع MX_DMA_Init

کلاک کنترل کننده نخست را روشن و وقفه کانال یک را با اولویت پیش دستی دو و زیر اولویت یک فعال می کند. تنظیم جهت، اندازه داده و حالت چرخشی در تابع پشتیبانی ADC انجام می شود.

۹.۱.۱۳ تابع MX_GPIO_Init

کلاک درگاه‌های A تا E را روشن، سطح اولیه خروجی‌های قدرت و خطوط یک‌سیمه را ایمن و سپس حالت پایه‌ها را تنظیم می‌کند. خطوط قدرت پوش پول کم‌سرعت‌اند. دو خط دما روی PB10 و PC7 از ابتدا خروجی تخلیه‌باز و در حالت آزاد بالا هستند؛ این تنظیم در فایل IOC نیز ثبت شده است. تنظیم اولیه بالا برای خروجی‌های فعال‌پایین مانع پالس روشن ناخواسته می‌شود.

۱۰.۱.۱۳ تابع Error_Handler

در نسخه مولد بدنه عملیاتی ندارد. بنابراین خطای راهاندازی فقط جریان همان تابع را برمی‌گرداند و سامانه ممکن است با پیرامونی ناقص ادامه دهد. تبدیل آن به خاموش‌سازی خروجی‌ها، ثبت علت و حلقه امن یا بازنشانی واچ‌داگ یک کار باز مهم است؛ پیاده‌سازی باید داخل ناحیه کاربر قرار گیرد.

۲.۱۳ توابع پشتیبانی پیرامونی

۱.۲.۱۳ تابع HAL_MspInit

کلاک رابط عملکرد جایگزین و توان را روشن می‌کند و رابط JTAG را غیرفعال ولی رابط SWD را فعال نگه می‌دارد. بنابراین پایه‌های برنامه‌ریزی دو سیمه در دسترس و برخی پایه‌های JTAG برای GPIO آزاد می‌شوند.

۲.۲.۱۳ توابع HAL_ADC_MspInit و HAL_ADC_MspDeInit

نسخه راهاندازی کلاک ADC و درگاه A را روشن، پایه‌های PA4/PA5 را آنالوگ، کانال نخست DMA را انتقال پیرامونی به حافظه با افزایش نشانی حافظه، داده نیم‌کلمه و حالت چرخشی می‌کند. دسته DMA به ADC پیوند و وقفه فعال می‌شود. نسخه توقف همه این منابع را آزاد می‌کند.

۳.۲.۱۳ توابع HAL_I2C_MspInit و HAL_I2C_MspDeInit

پایه‌های PB8/PB9 را خروجی عملکرد جایگزین تخلیه‌باز پرسرعت، بازنگاشت I2C را فعال و کلاک رابط را روشن می‌کنند. تابع توقف کلاک و پایه‌ها را آزاد می‌سازد.

۴.۲.۱۳ توابع HAL_SPI_MspInit و HAL_SPI_MspDeInit

برای رابط دوم، پایه کلاک و خروجی داده را عملکرد جایگزین پوش پول و ورودی داده را ورودی بدون کشش می کند. کلاک و وقفه SPI فعال می شوند. تابع توقف ترتیب معکوس را انجام می دهد.

۵.۲.۱۳ توابع پشتیبانی تایمرها

دو تابع HAL_TIM_Base_MspInit و HAL_TIM_Base_MspDeInit مسئول این بخش هستند. براساس نمونه تایمر، کلاک و وقفه تایمر یک، دو یا سه را فعال یا غیرفعال می کنند. اولویت تایمر سه از تایمر دو بالاتر و هر دو از تایمر یک بالاتر تنظیم شده است؛ پس از بازآرایی callbackها بسیار کوتاه اند.

۶.۲.۱۳ توابع HAL_UART_MspInit و HAL_UART_MspDeInit

کلاک رابط دوم و درگاه D را فعال، ارسال PD5 را عملکرد جایگزین و دریافت PD6 را ورودی می کند، بازنگاشت را فعال و وقفه UART را با اولویت بالا روشن می سازد. تابع توقف این تنظیمات را آزاد می کند.

۳.۱۳ هندلرهای استثنا و وقفه

توابع خطای سخت، مدیریت حافظه، خطای گذرگاه و دستور نامعتبر پس از ورود در حلقه بی نهایت می مانند. وقفه تیک سامانه شمارنده میلی ثانیه HAL را افزایش می دهد. وقفه های DMA، ADC، سه تایمر، SPI و UART فقط هندلر HAL متناظر را فراخوانی می کنند. هندلرهای سرویس سامانه، پایش اشکال و درخواست معوق بدنه کاربر ندارند. هیچ منطق محصولی نباید مستقیماً به این هندلرها بازگردانده شود.

۴.۱۳ فایل سامانه و راه انداز

فایل system_stm32f1xx.c متغیر کلاک هسته و تنظیمات پایه پیش از ورود به برنامه را فراهم می کند. فایل اسمبلی Keil جدول بردار، پشته، نقطه بازنشانی و انتقال به کتابخانه زمان اجرا را تعریف می کند. PlatformIO از نسخه متناظر چارچوب خود استفاده می کند و فایل سامانه محلی را برای جلوگیری از نماد تکراری کنار می گذارد.

فصل ۱۴

نشانگر زنده‌بودن و آزمون دو چراغ

۱.۱۴ هدف آزمون

دو چراغ متصل به پایه‌های PC13 و PC14 یک نشانگر مستقل و غیرمسدودکننده برای مشاهده اجرای برنامه فراهم می‌کنند. الگوی عادی فقط زمانی جلو می‌رود که تابع App_Process مرتب در حلقه اصلی فراخوانی شود. ثابت ماندن طولانی الگو، پس از در نظر گرفتن زمان طبیعی عملیات حسگرها، می‌تواند نشانه توقف حلقه اصلی یا گیرکردن یک تابع باشد. الگوی خطای سریع نیز ورود برنامه به Error_Handler را از اجرای عادی متمایز می‌کند.

این نشانگر جایگزین واچ‌داگ، ثبت خطا یا دیباگر نیست؛ چراغ فقط یک مشاهده سریع میدانی از زنده‌بودن حلقه اصلی ارائه می‌دهد.

۲.۱۴ اتصال الکتریکی و قطب فعال

هر چراغ باید مقاومت سری مستقل داشته باشد. برای کاهش بار پایه‌های دامنه پشتیبان درگاه سوم، مقاومت ۶۸۰ اهم تا یک کیلو اهم و چراغ کم‌جریان پیشنهاد می‌شود. اتصال پیش‌فرض نرم‌افزار فعال پایین است:

3.3 V -> series resistor -> LED anode -> LED cathode -> PC13 or PC14

در این اتصال، صفر منطقی چراغ را روشن می‌کند. اگر اتصال برد از خروجی پایه به مقاومت و سپس چراغ و زمین است، مقدار DIAGNOSTIC_LED_ACTIVE_LOW در فایل Application/Inc/diagnostic_led.h باید از یک به صفر تغییر کند. دو چراغ نباید یک مقاومت مشترک داشته باشند. زمین برد و پروگرامر نیز باید مشترک باشد.

۳.۱۴ تنظیمات مولد کد

در فایل `STM32F107.ioc` هر دو پایه به صورت خروجی پوش پول با سرعت پایین، بدون مقاومت کشش داخلی و سطح اولیه بالا ثبت شده اند. برچسب ها به ترتیب `DEBUG_LED_1` و `DEBUG_LED_2` هستند. منبع کلاک کم فرکانس خارجی فعال نیست؛ بنابراین استفاده از `PC14` به عنوان خروجی با تنظیم کلاک موجود تعارض ندارد.

ماژول عیب یابی هنگام راه اندازی همین تنظیم را دوباره و به صورت بی خطر اعمال می کند. این تکرار عمدی است تا اگر خطای راه اندازی پیش از پایان تابع تنظیم پایه ها رخ داد، امکان نمایش خطا باقی بماند. پس از هر تولید مجدد باید وجود دو پایه، برچسب ها، سطح اولیه بالا و سرعت پایین کنترل شود.

۴.۱۴ فایل ها و رابط عمومی

پیاده سازی در `Application/Src/diagnostic\_led.c` و رابط عمومی در `Application/Inc/` `diagnostic\_led.h` قرار دارد. فایل پیاده سازی مستقل از فایل های مولد است و به پروژه Keil نیز افزوده شده است؛ PlatformIO آن را به طور خودکار از پوشه سورس پیدا می کند.

۱.۴.۱۴ تابع `DiagnosticLed_Init`

ورودی و خروجی ندارد. کلاک درگاه GPIOC را فعال، دو پایه را خروجی پوش پول کم سرعت و هر دو چراغ را خاموش می کند. چند بار فراخوانی آن مجاز است، ولی هر بار حالت و گام الگو را بازنشانی می کند.

۲.۴.۱۴ تابع `DiagnosticLed_SetMode`

یک مقدار از نوع `DiagnosticLed_Mode` می گیرد، الگوی جدید را انتخاب و اجرای آن را از گام نخست آغاز می کند. مقدار خارج از محدوده به حالت خاموش تبدیل می شود. این تابع برای متن اصلی طراحی شده و نباید همزمان از وقفه تغییر داده شود.

۳.۴.۱۴ تابع `DiagnosticLed_Process`

ورودی و خروجی ندارد و باید با بیشترین تکرار ممکن در حلقه اصلی صدا زده شود. زمان را با `HAL_GetTick` می خواند و با تفریق بدون علامت، عبور شمارنده میلی ثانیه از بیشینه را نیز درست مدیریت می کند. اگر موعد گام بعد نرسیده باشد بلافاصله بازمی گردد؛ در نتیجه هیچ تأخیر مسدود کننده ای به سامانه تحمیل نمی شود.

اگر حلقه اصلی برای چند گام متوقف شود، تابع در اولین اجرای بعدی تعداد گام‌های گذشته را محاسبه و الگو را به موقعیت درست می‌برد. این رفتار از انباشته شدن تأخیر زمانی جلوگیری می‌کند.

۴.۴.۱۴ تابع DiagnosticLed_GetMode

ورودی ندارد و حالت فعال را برمی‌گرداند. این تابع برای مشاهده در دیباگر، گزارش وضعیت یا توسعه فرمان‌های عیب‌یابی آینده در نظر گرفته شده است و سخت‌افزار را تغییر نمی‌دهد.

۵.۱۴ الگوهای قابل انتخاب

۱.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_OFF

هر دو چراغ خاموش می‌شوند. این حالت برای غیرفعال کردن آزمون بدون حذف کد است.

۲.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_HEARTBEAT

دو تپ کوتاه و سپس مکث ایجاد می‌شود. این الگو نشانگر آرام برای بهره‌برداری دائمی است.

۳.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_DANCE

چراغ اول، چراغ دوم، هر دو و سپس مکث اجرا می‌شود. چرخه تقریباً یک ثانیه است و حالت پیش‌فرض برنامه محسوب می‌شود.

۴.۵.۱۴ حالت DIAGNOSTIC_LED_MODE_ERROR

دو چراغ هر ۱۵۰ میلی‌ثانیه به سرعت جابه‌جا می‌شوند. این الگو نشان‌دهنده ورود به خطای بحرانی است.

۶.۱۴ اتصال به چرخه برنامه

تابع App_Init ماژول را راه‌اندازی و حالت رقص را انتخاب می‌کند. تابع App_Process پیش از رسیدگی به ارتباط و نمونه‌برداری، یک بار تابع پردازش چراغ را اجرا می‌کند. اگر برنامه وارد Error_Handler شود، دو پایه

دوباره راه اندازی و حالت خطا در یک حلقه اختصاصی پردازش می شود.

برای استفاده آرام تر پس از پایان آزمون کافی است در Application/Src/app.c حالت اولیه از رقص به ضربان تغییر کند؛ حذف فراخوانی پردازش توصیه نمی شود، زیرا ارزش نشانگر زنده بودن از بین می رود.

۷.۱۴ پروگرام با رابط چهار سیمه

برای پروگرام معمولی اتصال های زمین مشترک، ولتاژ مرجع ۳.۳ ولت، داده رابط و کلاک رابط لازم اند. نام دو خط رابط به ترتیب SWDIO روی PA13 و SWCLK روی PA14 است. در صورت دشواری اتصال، خط بازنشانی نیز می تواند به عنوان سیم پنجم افزوده شود.

پس از اتصال J-Link و روشن کردن هدف، فرمان های زیر خروجی را ساخته و پروگرام می کنند:

```
pio run
pio run --target upload
```

در Keil نیز هدف STM32F107 باز، پروژه ساخته و گزینه بارگذاری انتخاب می شود. فایل MDK-ARM/ STM32F107.uvprojx از قبل مازول چراغ را در گروه کد کاربر دارد.

۸.۱۴ نتیجه پروگرام انجام شده

در تاریخ ۱۴۰۵/۰۶/۲۳، بارگذاری مستقیم از همین پروژه با PlatformIO و J-Link انجام شد. پروگرامر از طریق رابط USB شناسایی و ولتاژ مرجع هدف ۳۲۵.۳ ولت گزارش شد. قطعه STM32F107VC و هسته Cortex-M3 از مسیر SWD با موفقیت شناسایی شدند. پاک کردن، نوشتن، تطبیق محتوای حافظه و بازنشانی نهایی همگی بدون خطا پایان یافتند. بنابراین فایل اجرایی این نسخه روی سخت افزار بارگذاری شده است؛ مشاهده فیزیکی ترتیب چراغ ها باید توسط کاربر روی برد تأیید شود.

۹.۱۴ روند آزمون و تفسیر نتیجه

ابتدا برای جلوگیری از عملکرد ناخواسته، بارهای قدرت رله، هیتر و سرمایش قطع یا ایمن شوند. سپس برنامه بارگذاری و برد بازنشانی شود.

۱. در ثانیه نخست باید ترتیب چراغ اول، چراغ دوم و هر دو مشاهده شود.

۲. ادامه منظم چرخه نشان می دهد حلقه اصلی در حال اجرا و تیک سامانه فعال است.
 ۳. مکث یا پرش کوتاه باید با دیباگر و شمار خطای ارتباط ADE7880 زمان سنجی و ریشه یابی شود.
 ۴. ثابت شدن طولانی الگو یعنی حلقه اصلی دیگر تابع پردازش را اجرا نمی کند یا تیک سامانه متوقف شده است.
 ۵. تناوب سریع و پیوسته چراغ ها یعنی برنامه وارد هندلر خطای بحرانی شده است.
 ۶. خاموش بودن دائمی هر دو چراغ می تواند از قطب اشتباه، سیم بندی نادرست، نبود مقاومت و تغذیه، پروگرام نشدن یا اجرا نشدن برنامه ناشی شود.
- این آزمون فقط راه اندازی اولیه و زنده بودن نرم افزار را نشان می دهد. تأیید ADE7880، کالیبراسیون و بازیابی ارتباط باید مطابق چک لیست فصل بعد انجام شود. سنسورها، ارتباط سریال، حافظه و کنترل بارها مربوط به مراحل توسعه بعدی اند و در نسخه فعال فعلی اجرا نمی شوند.

فصل ۱۵

راه‌اندازی ارتباط و اندازه‌گیری با ADE7880

۱.۱۵ دامنه این مرحله و نتیجه قابل انتظار

در این مرحله رابط SPI2 میکروکنترلر دوباره فعال، کتابخانه ADE7880 مستقل از خانواده میکروکنترلر بازنویسی و خواندن دوره‌ای کمیت‌های اصلی به برنامه افزوده شد. برنامه در هر نمونه، ولتاژ مؤثر، جریان مؤثر، توان اکتیو، توان ظاهری و ضریب توان هر سه فاز و همچنین جریان مؤثر نول را می‌خواند. داده خام کامل در متغیر `g_app_electrical` منتشر می‌شود و در پنجره Watch قابل مشاهده است. علاوه بر داده خام، مقدارهای مهندسی هر سه فاز با جدول ضرایب پیش‌فرض برد در آرایه‌های متناظر `voltage_v` قرار می‌گیرد. معتبر بودن هر خانه و منبع ضریب آن به ترتیب در `voltage_valid` و `voltage_source` ثبت می‌شود.

خواندن رجیستر با تبدیل مقدار خام به واحد فیزیکی یکسان نیست. ضریب توان از قالب ثابت خود تراشه مستقیماً به بازه منفی یک تا مثبت یک تبدیل پذیر است، اما تبدیل دیگر کمیت‌ها به ولت، آمپر و وات وابسته به نسبت ترانس جریان، شبکه تقسیم ولتاژ، مقاومت بار، بهره ورودی و کالیبراسیون واقعی همان برد است. جدول پیش‌فرض برای راه‌اندازی و مشاهده هر سه فاز مفید است، ولی تا آزمون با مرجع معتبر نباید مبنای حفاظت، ایمنی یا اندازه‌گیری نهایی قرار گیرد.

۲.۱۵ منبع بازیابی اتصال‌های قبلی

مخزن هنگام آغاز این مرحله هیچ تاریخچه Git نداشت، ولی فایل پشتیبان قدیمی همین پروژه در مسیر بیرونی `embedded/software/backup/Main/STM32F107.ioc` هنوز موجود بود. تنظیمات آن با گزارش قبلی و سورس قدیمی تطبیق داده شد. هر سه منبع اتصال زیر را تأیید کردند:

پایه میکرو	سیگنال	تنظیم لازم
PB12	ADE_CS / SS	خروجی پوش‌پول، فعال‌پایین، سطح بی‌کار بالا
PB13	SPI2_SCK	خروجی عملکرد جایگزین پوش‌پول
PB14	SPI2_MISO	ورودی بدون کشش داخلی
PB15	SPI2_MOSI	خروجی عملکرد جایگزین پوش‌پول

در فایل‌های قدیمی هیچ پایه‌ای برای RESET یا IRQ1 تراشه تعریف نشده بود. به همین دلیل نسخه حاضر از بازنشانی نرم‌افزاری و بررسی رجیستر STATUS1.RSTDONE استفاده می‌کند. اگر در شماتیک واقعی این دو خط به میکرو متصل‌اند، شماره پایه‌ها باید در مرحله بعد از روی شماتیک قطعی شود؛ تا آن زمان نباید پایه‌ای برای آن‌ها حدس زده شود.

۳.۱۵ کنترل الزامات دیتاشیت

مرجع این پیاده‌سازی دیتاشیت رسمی بازبینی C شرکت Analog Devices است: [ADE7880 Rev. C](#)

نکات زیر مستقیماً در طراحی رعایت شده‌اند:

- تراشه پس از روشن‌شدن تقریباً ۴۰ میلی‌ثانیه برای آماده‌شدن نیاز دارد؛ کتابخانه ۵۰ میلی‌ثانیه حاشیه می‌دهد.
- رابط پیش‌فرض پس از reset از نوع I2C است. سه گذار پایه SS از بالا به پایین رابط را به SPI تغییر می‌دهد.
- نوشتن رجیستر هشت‌بیتی CONFIG2 پس از انتخاب SPI، انتخاب رابط را قفل می‌کند.
- داده روی لبه بالارونده SCLK نمونه‌برداری و پرارزش‌ترین بیت ابتدا منتقل می‌شود.
- بیشینه کلاک مجاز SPI برابر ۵.۲ مگاهرتز است. نرخ انتخاب‌شده ۵.۵۶۲ کیلوبیت‌برثانیه حاشیه کافی دارد.
- پایان reset نرم‌افزاری با دو شرط کنترل می‌شود: SWRST=0 و RSTDONE=1.
- پردازشگر داخلی تنها پس از نوشتن و بازخوانی موفق مقدار یک در رجیستر RUN شروع می‌شود.

۴.۱۵ تنظیم از صفر در STM32CubeMX

فایل STM32F107.ioc مرجع تنظیم مولد است. برای بازسازی دستی همین پیکربندی مراحل زیر انجام شود:

۱. فایل پروژه را در نسخه سازگار STM32CubeMX باز کنید و قطعه STM32F107VCTx با بسته LQFP100 را تغییر ندهید.

۲. از بخش Connectivity، رابط SPI2 را روی Full-Duplex Master قرار دهید.

۳. کنترل کنید PB13 برابر SPI2_SCK، PB14 برابر SPI2_MISO و PB15 برابر SPI2_MOSI شده باشد.

۴. پایه PB12 را GPIO Output با برچسب ADE_CS انتخاب کنید. سطح اولیه باید High، نوع خروجی Push-Pull، کشش No pull و سرعت High باشد.

۵. در تنظیم‌های SPI2 حالت ارباب، جهت دوخطه، طول هشت بیت، CPHA=2 Edge، CPOL=High، انتخاب تراشه نرم‌افزاری، MSB First و تقسیم‌کننده ۶۴ را انتخاب کنید.

۶. مقدار محاسبه‌شده کلاک باید با کلاک گذرگاه اول ۳۶ مگاهرتز برابر ۵.۵۶۲ کیلو بیت بر ثانیه باشد.

۷. فعال کردن وقفه SPI2 برای روش polling این مرحله لازم نیست. افزودن بی دلیل وقفه فقط پیچیدگی ایجاد می‌کند.

۸. در Project Manager > Code Generator گزینه نگهداری کد کاربر فعال و حذف فایل‌های قبلی غیرفعال باقی بماند.

۹. کد را تولید و سپس تفاوت‌ها را با Git بازبینی کنید. وجود MX_SPI2_Init و تنظیم پایه‌ها نتیجه مورد انتظار مولد است.

فراخوانی‌های برنامه داخل ناحیه‌های حفاظت‌شده فایل اصلی هستند:

```

۱ /* USER CODE BEGIN 2 */
۲ App_Init(&hspi2);
۳ /* USER CODE END 2 */
۴ /* USER CODE BEGIN WHILE */
۵ while (1) {
۶     App_Process();
۷ /* USER CODE END WHILE */
۸ }

```

در نتیجه، تولید مجدد CubeMX ممکن است بدنه‌های مولد MX_SPI2_Init و HAL_SPI_MspInit را بازسازی کند، اما منطق کتابخانه و برنامه در پوشه‌های Libraries/ADE7880 و Application پاک نمی‌شود و دو نقطه اتصال نیز داخل بلوک کاربر حفظ می‌شوند.

۵.۱۵ معماری کتابخانه قابل استفاده مجدد

کتابخانه فقط دو فایل دارد:

○ Libraries/ADE7880/Inc/ade7880.h: انواع، رجیسترها و رابط عمومی؛

○ Libraries/ADE7880/Src/ade7880.c: پروتکل، reset، خواندن و کالیبراسیون.

کتابخانه هدر stm32f1xx_hal.h را نیاز ندارد. وابستگی سخت‌افزار با ساختار ADE7880_Transport به آن تزریق می‌شود:

```

۱ typedef struct {
۲     void *context;
۳     ADE7880_Status (*write)(void *, const uint8_t *, size_t, uint32_t);
۴     ADE7880_Status (*read)(void *, uint8_t *, size_t, uint32_t);
۵     void (*select)(void *, bool active);
۶     void (*delay_ms)(void *, uint32_t);
۷ } ADE7880_Transport;
```

برای انتقال کتابخانه به میکرو دیگر فقط همین چهار callback پیاده می‌شود. برای مثال در STM32، callback نوشتن به HAL_SPI_Transmit و callback خواندن به HAL_SPI_Receive وصل شده است. در پروژه دیگر می‌توان آن‌ها را به درایور LL، ESP-IDF، Zephyr یا یک SPI نرم‌افزاری متصل کرد، بدون آن‌که فایل کتابخانه تغییر کند.

راهنمای مستقل و کوتاه استفاده از کتابخانه نیز در فایل Libraries/ADE7880/README.md قرار دارد؛ بنابراین برای انتقال ماژول به پروژه دیگر لازم نیست گزارش کامل محصول همراه آن کپی شود.

۶.۱۵ قرارداد کامنت‌گذاری و روش خواندن کد

تمام تعریف‌های عمومی کتابخانه در ade7880.h با قالب Doxygen انگلیسی مستند شده‌اند. هر تابع عمومی باید هدف، پارامترهای ورودی، خروجی و خطاهای قابل انتظار را داشته باشد. اعضای ساختار، مقدارهای enum

و رجیسترها نیز کنار تعریف خود توضیح انگلیسی دارند. فایل پیاده‌سازی علاوه بر توضیح هر تابع، دلیل مراحل غیر بدیهی مانند ترتیب بایت، بسط علامت، جلوگیری از انتشار نمونه ناقص و پاک کردن RSTDONE را ثبت می‌کند.

قاعده ادامه پروژه چنین است:

- نام تابع و متغیر باید هدف را بیان کند و کامنت جای نام‌گذاری نامناسب را نگیرد؛
- رابط عمومی با @brief، @param و @return انگلیسی مستند شود؛
- عدد ثابت سخت‌افزاری باید واحد و علت داشته باشد؛
- کامنت باید دلیل یا قرارداد را توضیح دهد، نه آن‌که همان دستور ساده را تکرار کند؛
- هر تغییر رفتار در گزارش فارسی و فایل وضعیت نیز ثبت شود.

۷.۱۵ مرجع سریع توابع کتابخانه

- ADE7880_Init: ثبت و کنترل چهار callback سخت‌افزار؛ هنوز با تراشه تبادل انجام نمی‌دهد.
- ADE7880_Begin: انتخاب SPI، بازنشانی نرم‌افزاری، انتظار محدود و شروع DSP.
- ADE7880_ReadRegister: خواندن عمومی رجیسترهای ۸، ۱۶ یا ۳۲ بیتی.
- ADE7880_WriteRegisterVerified: نوشتن رجیستر و مقایسه بازخوانی برای تشخیص خطا.
- ADE7880_ReadPhase: خواندن همه کمیت‌های خام یک فاز.
- ADE7880_ReadAll: ساخت نمونه کامل سه فاز و نول؛ در خطا خروجی نیمه‌کاره منتشر نمی‌شود.
- ADE7880_CalculateLinearCalibration: محاسبه scale/offset از دو نقطه خام و مرجع.
- ADE7880_ApplyLinearCalibration: تبدیل یک شمار خام به واحد مهندسی.
- ADE7880_Convert: تبدیل یک نمونه کامل با مجموعه ضرایب همه کانال‌ها.
- App_SetPhaseCalibration: جایگزینی مستقل ضریب ولتاژ، جریان، توان اکتیو یا توان ظاهری هر فاز.
- App_CalibratePhaseMultiPoint: برازش و نصب کالیبراسیون چندنقطه‌ای برای یک کمیت و یک فاز.
- App_CalibrateNeutralCurrentMultiPoint: برازش و نصب مستقل جریان نول.

○ App_ResetCalibrationToDefaults: حذف ضرایب زمان اجرا و بازگرداندن جدول آغاز کار.

ساده‌ترین مسیر استفاده در همین پروژه فقط دو فراخوانی دارد: یک بار App_Init(&hspi2) بعد از راهاندازی SPI2 و سپس اجرای پیوسته App_Process() در حلقه اصلی. لایه برنامه ساختار انتقال، بازیابی خطا، زمان نمونه‌برداری و تبدیل ولتاژ را مدیریت می‌کند؛ کاربر معمولاً فقط g_app_electrical را می‌خواند.

۸.۱۵ توالی راهاندازی برنامه

تابع App_Init پس از راهاندازی پایه‌ها و SPI2 فراخوانی می‌شود. توالی دقیق آن چنین است:

۱. ماژول دو LED راهاندازی می‌شود.
 ۲. callback های HAL در یک ADE7880_Transport قرار می‌گیرند.
 ۳. ADE7880_Init اعتبار callback ها را بررسی و CS را غیرفعال می‌کند.
 ۴. ADE7880_Begin پنجاه میلی‌ثانیه صبر، سه پالس CS ایجاد و SPI را با CONFIG2 قفل می‌کند.
 ۵. reset نرم‌افزاری با مهلت ۱۰۰ میلی‌ثانیه اجرا می‌شود؛ حلقه نامحدود وجود ندارد.
 ۶. رجیستر RUN نوشته و بازخوانی می‌شود.
 ۷. رجیستر هشتمین VERSION خوانده و در وضعیت دیباگ ثبت می‌شود.
 ۸. موفقیت حالت رقص LED و خطا تناوب سریع دو LED را فعال می‌کند.
- اگر ارتباط قطع باشد، برنامه در راهاندازی گیر نمی‌کند. وضعیت آفلاین ثبت و هر دو ثانیه تلاش مجدد انجام می‌شود. پس از بازیابی ارتباط، الگوی LED خودکار به رقص برمی‌گردد. در حالت سالم یک snapshot کامل تقریباً هر یک ثانیه خوانده می‌شود.

۹.۱۵ رجیسترها و قالب داده خوانده‌شده

کمیت	فاز A	فازهای B/C	تفسیر
جریان مؤثر	0x43C0	0x43C2/0x43C4	مقدار ۲۴ بیتی بدون علامت کاربردی
ولتاژ مؤثر	0x43C1	0x43C3/0x43C5	مقدار ۲۴ بیتی بدون علامت کاربردی

کمیت	فاز A	فازهای B/C	تفسیر
توان اکتیو	0xE513	0xE514/0xE515	مقدار ۲۴ بیتی علامت‌دار با بسط علامت
توان ظاهری	0xE519	0xE51A/0xE51B	داده ۲۴ بیتی در کلمه ۳۲ بیتی
ضریب توان	0xE902	0xE903/0xE904	عدد علامت‌دار Q1.15؛ هر LSB برابر 2^{-15}
جریان نول	0x43C6	ندارد	مقدار ۲۴ بیتی

در نسخه قدیمی ضریب توان از رجیستر هارمونیک مشترک FPF خوانده می‌شد و انتخاب فاز نیز درست تضمین نشده بود. نسخه حاضر مستقیماً از سه رجیستر APF/BPF/CPF استفاده می‌کند؛ بنابراین هم ساده‌تر و هم مطابق بخش Power Factor دیتاشیت است.

۱۰.۱۵ خواندن در VS Code و PlatformIO

پس از بازکردن ریشه پروژه، افزونه PlatformIO IDE را نصب و محیط stm32f107vc_jlink را انتخاب کنید. ساخت از ترمینال:

```
pio run -e stm32f107vc_jlink
```

اگر فرمان pio در PATH نیست، روی ویندوز می‌توان فایل اجرایی محیط خود PlatformIO را اجرا کرد:

```
%USERPROFILE%\platformio\penv\Scripts\platformio.exe run \
-e stm32f107vc_jlink
```

برای پروگرام:

```
pio run -e stm32f107vc_jlink --target upload
```

برای دیباگ، breakpoint را بعد از انتساب g_app_electrical.raw در App_Process قرار دهید و متغیر g_app_electrical را به Watch اضافه کنید. نتیجه سالم باید چنین ویژگی‌هایی داشته باشد:

◦ online=true

- `last_status=ADE7880_STATUS_OK`؛
- `die_version` دارای مقدار ثابت و تکرارپذیر؛
- افزایش `successful_samples` تقریباً هر ثانیه؛
- تازه‌شدن `updated_at_ms`؛
- ثابت‌ماندن `communication_errors` در صفر.

۱۱.۱۵ ساخت و دیباگ در Keil MDK

فایل `MDK-ARM/STM32F107.uvprojx` باز شود. کتابخانه از قبل در گروه `Libraries/Reusable` عضو است و مسیر `include` آن نیز ثبت شده است. بعد از تولید مجدد `CubeMX` موارد زیر کنترل شود:

۱. فایل‌های `app.c` و `diagnostic_led.c` از پوشه `Application/Src` هنوز عضو پروژه باشند.

۲. فایل `Libraries/ADE7880/Src/ade7880.c` فقط یک بار عضو پروژه باشد.

۳. مسیرهای `Application/Inc` و `Libraries/ADE7880/Inc` در `Include Paths` باشند.

۴. نمادهای `USE_HAL_DRIVER` و `STM32F107xC` تعریف شده باشند.

۵. هدف با گزینه `Build Target` بدون `warning` ساخته شود.

در پنجره `Watch` همان متغیر `g_app_electrical` اضافه شود. برای جلوگیری از توقف طولانی سیستم قدرت، `breakpoint` تنها هنگام جدا بودن بارها استفاده شود. اگر برنامه باید آزاد اجرا شود، از `Watch` با به‌روزرسانی دوره‌ای و بدون توقف مکرر بهره بگیرید.

ساخت واقعی همین پروژه با Keil و `Arm Compiler 6.24` اجرا شد و با نتیجه `0 Error(s), 0 Warning(s)` پایان یافت. اندازه‌های گزارش شده برابر `Code=9628`، `RO-data=476`، `RW-data=12` و `ZI-data=1460` بایت بودند و فایل `HEX` نیز تولید شد.

خطاهای اولیه همگی از ناشناخته‌بودن `DMA_HandleTypeDef` آغاز می‌شدند. فایل پیاده‌سازی `DMA` مربوط به `HAL` در پروژه وجود داشت، ولی تعریف `HAL_DMA_MODULE_ENABLED` در `Core/Inc/stm32f1xx_hal_conf.h` غیرفعال بود. با فعال کردن این تعریف، نوع‌های مورد نیاز `handle`های `I2C/SPI/TIM/UART` پیش از استفاده در هدرهای `HAL` تعریف شدند. این یک وابستگی تعریفی در `HAL` است و به‌تنهایی به معنی استفاده منطقی برنامه از انتقال `DMA` نیست.

۱۲.۱۵ کالیبراسیون دو نقطه‌ای

نوع ADE7880_LinearCalibration دو عدد scale و offset دارد و رابطه زیر را اعمال می‌کند:

$$\text{engineering} = \text{raw} * \text{scale} + \text{offset}$$

برای نمونه، فرض کنید در دو نقطه مرجع ۱۰۰ و ۲۳۰ ولت، رجیستر ولتاژ به ترتیب ۱۱۰۰۰۰۰ و ۲۵۵۰۰۰۰ باشد. محاسبه ضرایب بدون دست‌کاری دستی فرمول چنین است:

```

۱ ADE7880_LinearCalibration voltage_a;
۲
۳ if (ADE7880_CalculateLinearCalibration(
۴     1100000.0F, 100.0F,
۵     2550000.0F, 230.0F,
۶     &voltage_a) == ADE7880_STATUS_OK) {
۷     App_SetVoltageCalibration(ADE7880_PHASE_A, &voltage_a);
۸ }

```

سپس ADE7880_Convert یک snapshot خام و ساختار ضرایب را به مقادیر مهندسی تبدیل می‌کند. بهتر است ضرایب هر فاز مستقل باشند. روند عملی کالیبراسیون:

۱. برد و مرجع اندازه‌گیری حداقل تا رسیدن دما و تغذیه به حالت پایدار روشن بمانند.
۲. برای هر فاز دو یا چند نقطه دور از هم انتخاب شود؛ نقطه صفر به تنهایی برای RMS مناسب نیست.
۳. در هر نقطه میانگین چند ده snapshot خام ثبت شود.
۴. مقدار مرجع کالیبره‌شده هم‌زمان ثبت و scale/offset محاسبه شود.
۵. ضرایب با شماره سریال برد، تاریخ، ابزار مرجع و شرایط آزمون ذخیره شوند.
۶. پس از اعمال ضرایب، نقطه سوم که در محاسبه استفاده نشده کنترل شود.

۱.۱۲.۱۵ ضرایب ولتاژ بازیابی‌شده و خروجی فعلی

در سورس قدیمی همین برد تبدیل زیر پیدا شد:

$$\text{phase A volts} = \text{AVRMS} * 0.00055963 * 1.017$$

$$\text{phase B volts} = \text{BVRMS} * 0.00055963$$

این دو ضریب اکنون هنگام App_Init بارگذاری می‌شوند. پس از نخستین نمونه موفق، خروجی‌های زیر در Watch قابل استفاده‌اند:

○ voltage_v[0/1/2]: ولتاژ تقریبی فازهای A/B/C؛

○ voltage_valid[0/1/2]=true: تبدیل جدول پیش‌فرض پس از نخستین نمونه کامل انجام شده است؛

○ voltage_source[0/1/2]: مقدار APP_CALIBRATION_DEFAULT تا زمانی که ضریب اندازه‌گیری‌شده نصب نشده باشد؛

○ ضریب اولیه فاز C از فاز B کپی شده و باید روی سخت‌افزار واقعی کنترل شود.

با فراخوانی App_SetVoltageCalibration منبع ضریب همان فاز «کاربر» ثبت می‌شود و نمونه بعدی با ضریب جدید منتشر خواهد شد. برای تکمیل کالیبراسیون جریان و توان هنوز نسبت CT، مقدار مقاومت burden، شبکه ورودی و محدوده نامی لازم است.

۲.۱۲.۱۵ مشاهده جریان، توان و ضریب توان در دیباگر

دراپور از ابتدا رجیسترهای جریان RMS، توان اکتیو، توان ظاهری و ضریب توان را نیز در هر snapshot می‌خواند. این داده‌ها در ساختار خام زیر قرار دارند:

○ raw.phase[0] برای فاز A؛

○ raw.phase[1] برای فاز B؛

○ raw.phase[2] برای فاز C؛

○ raw.neutral_current_rms برای جریان نول.

برای نمونه، پیش از انجام هر کالیبراسیون می‌توان در Watch ساختار g_app_electrical.raw.phase[0] را باز کرد و اعضای current_rms، active_power، apparent_power و power_factor_q15 را برای فاز A دید. این مقادیر به‌جز ضریب توان، شمار خام پردازشگر داخلی تراشه هستند و واحد آمپر یا وات ندارند.

نسخه جدید برنامه فیلدهای مستقیم و خوانای زیر را نیز منتشر می‌کند:

معنی و شرط استفاده	فیلد در g_app_electrical
آرایه جریان RMS برحسب آمپر.	current_a
آرایه توان اکتیو برحسب وات.	active_power_w
آرایه توان ظاهری برحسب ولت‌آمپر.	apparent_power_va
آرایه ضریب توان علامت‌دار در بازه -۱ تا +۱.	power_factor
جریان نول برحسب آمپر؛ فقط با پرچم اعتبار متناظر قابل استفاده است.	neutral_current_a

آرایه power_factor_abs بزرگی ضریب توان را در بازه صفر تا یک و آرایه current_polarity جهت جبران CT هر فاز را به صورت NORMAL یا REVERSED نشان می‌دهد.

سه عضو هر آرایه به ترتیب فازهای A/B/C هستند. برای جریان، توان اکتیو و توان ظاهری به ترتیب پرچم‌های زیر کنترل شوند:

○ current_valid

○ active_power_valid

○ apparent_power_valid

ضریب توان پس از یک snapshot کامل با power_factor_valid=true قابل استفاده است.

رجیسترهای APF/BPF/CPF اعداد علامت‌دار Q1.15 هستند؛ بنابراین هر LSB وزن 2^{-15} دارد و برنامه آن‌ها را با تقسیم بر ۳۲۷۶۸ به ضریب توان تبدیل می‌کند. این تبدیل به نسبت CT یا شبکه ولتاژ وابسته نیست. علامت PF مطابق قرارداد تراشه اطلاعات پیش‌فاز/پس‌فاز بودن را نیز حمل می‌کند.

مقدار آمپر و وات به سخت‌افزار همان برد وابسته است. در نسخه نخست این بخش، چون ضریب کالیبراسیون جریان و توان بارگذاری نشده بود، برنامه این فیلدها را عمداً صفر و پرچم اعتبارشان را نادرست نگه می‌داشت؛ بنابراین تغییر داده خام همراه با صفر ماندن مقدار مهندسی نشانه خرابی SPI نبود.

در بازبینی سورس پشتیبان همین برد، ضرایب زیر برای فازهای A/B پیدا شدند. کد قدیمی نتیجه را در متغیر صحیح ۱۶ بیتی و بسته ارتباطی قرار می‌داد. آزمون واقعی با لامپ حدود ۲۰ وات نشان داد خروجی قدیمی جریان برحسب صدم آمپر و خروجی توان برحسب دهم وات بوده است: عددهای ۱۷ و ۲۳۱ به ترتیب حدود ۰.۱۷ A و ۲۳.۱ W هستند. بنابراین تبدیل درست به واحدهای SI چنین است:

$$\text{current amperes} = \text{IRMS} * (0.00036565 / 100)$$

$$\text{active watts} = \text{WATT} * (0.0170276 / 10)$$

فاز A در توان اصلاح اضافی 1.02 داشت. این ضرایب اکنون هنگام App_Init بارگذاری می شوند. برای کامل بودن راه اندازی سه فاز، فاز C فعلاً از مقادیر فاز B استفاده می کند و جریان نول از ضریب جریان فازها آغاز می شود؛ بنابراین پس از نخستین نمونه موفق همه کانال ها خروجی مهندسی و پرچم اعتبار دارند. همه ضرایب آغاز کار با APP_CALIBRATION_DEFAULT علامت گذاری می شوند. این پرچم فقط موفقیت تبدیل را نشان می دهد و تأیید دقت ضریب جایگزین موقت نیست.

کد قدیمی توان ظاهری را به واحد مهندسی تبدیل نمی کرد. دیتاشیت ADE7880 و راهنمای AN-1171 بیان می کنند بهره توان های اکتیو و ظاهری هر فاز در تراشه با هم تطبیق داده شده است؛ از این رو مقدار اولیه VA هر فاز با ضریب توان همان فاز آغاز می شود. وجود خانه مستقل apparent_power[phase] اجازه می دهد بعداً بدون اثر روی توان اکتیو، همان کانال جداگانه کالیبره شود.

برای تشخیص منشأ هر خروجی، آرایه های voltage_source, current_source, active_power_source و apparent_power_source در Watch قابل مشاهده اند. مقدار APP_CALIBRATION_USER یعنی ضریب پیش فرض همان خانه با کالیبراسیون اندازه گیری شده جایگزین شده است.

۳.۱۲.۱۵ جهت CT، علامت توان و علامت ضریب توان

اگر جهت اولیه یا ثانویه CT برعکس شود، بردار جریان ۱۸۰ درجه تغییر می کند. در نتیجه علامت توان اکتیو و PF علامت دار عوض می شود، ولی جریان RMS، توان ظاهری و بزرگی PF نباید تغییر کنند. برای نصب فقط مصرف کننده، جهت درست جهتی است که در هنگام روشن بودن بار، توان اکتیو مثبت باشد. اگر روی CT علامت وجود دارد، معمولاً P1/K سمت منبع، P2/L سمت بار، S1/k سمت ورودی جریان مثبت و S2/l سمت ورودی منفی قرار می گیرد؛ نام نهایی پایه ها حتماً باید با شماتیک همان برد و دیتاشیت سازنده CT تطبیق داده شود.

علامت PF را نباید برای انتخاب جهت CT به تنهایی استفاده کرد. طبق قرارداد ADE7880، علامت این رجیستر اطلاعات پیش فاز/پس فاز را نیز حمل می کند. بنابراین یک لامپ LED با ورودی یکسوساز و خازن می تواند با وجود توان مصرفی مثبت، PF علامت دار منفی داشته باشد. فیلد power_factor علامت را حفظ می کند و فیلد جدید power_factor_abs فقط بزرگی صفر تا یک را برای نمایش معمول ارائه می دهد.

اگر اصلاح فیزیکی جهت ممکن نباشد، پس از App_Init یک بار تنظیم زیر انجام می شود:

```

۱ (void)App_SetCurrentPolarity(
۲     ADE7880_PHASE_A,
۳     APP_CURRENT_POLARITY_REVERSED);

```

این تنظیم علامت توان اکتیو و PF علامت‌دار را اصلاح می‌کند؛ جریان RMS، توان ظاهری و قدرمطلق ضریب توان بدون تغییر می‌مانند. تشخیص و وارونه‌سازی خودکار در هر نمونه انجام نمی‌شود، زیرا در یک سامانه دوطرفه توان منفی می‌تواند صادرات واقعی باشد. مقدار انتخاب‌شده در آرایه current_polarity قابل مشاهده است و باید یک‌بار هنگام راه‌اندازی هر فاز تعیین شود.

هشدار ایمنی: ثانویه CT در حالی که جریان از اولیه عبور می‌کند هرگز باز نشود؛ امکان ایجاد ولتاژ خطرناک وجود دارد. تغییر سیم‌بندی فقط پس از بی‌برق کردن اولیه یا با روش اتصال کوتاه ایمن مورد تأیید سازنده انجام شود.

۴.۱۲.۱۵ واحد مهندسی و نمایش سازگار با برنامه قدیمی

برای جلوگیری از ابهام، فیلدهای محاسباتی همیشه واحد مهندسی صریح دارند:

○ current_a: آمپر؛

○ active_power_w: وات؛

○ apparent_power_va: ولت‌آمپر.

سه فیلد مستقل زیر فقط برای نمایش و پروتکل‌های fixed-point در نظر گرفته شده‌اند:

○ current_display: پیش‌فرض صدم آمپر؛ بنابراین 0.17 A به شکل 17 دیده می‌شود؛

○ active_power_display: پیش‌فرض دهم وات؛ بنابراین 23.1 W به شکل 231 دیده می‌شود؛

○ apparent_power_display: پیش‌فرض دهم ولت‌آمپر.

دو ثابت زیر در app.h پیش‌فرض را مشخص می‌کنند؛ مقدارهای آن‌ها به ترتیب ۱۰ و ۱۰۰ هستند:

```
۱ APP_DEFAULT_CURRENT_DISPLAY_UNITS_PER_AMP
۲ APP_DEFAULT_POWER_DISPLAY_UNITS_PER_WATT
```

تغییر موقت در زمان اجرا نیز بدون دست‌زدن به کالیبراسیون ممکن است:

```
۱ /* Make optional display fields use direct A and W units. */
۲ (void)App_SetDisplayUnits(1.0F, 1.0F);
```

۵.۱۲.۱۵ جدول واحد ضرایب پیش‌فرض هر سه فاز

همه ضرایب آغاز کار برد در یک متغیر ثابت و خوانا به نام `default_calibration` در ابتدای فایل `app.c` قرار گرفته‌اند. ساختار هر خانه دقیقاً $\{scale, offset\}$ و رابطه تبدیل برای همه کمیت‌ها یکسان است:

$$engineering = raw * scale + offset$$

کمیت	کانال	scale پیش‌فرض	offset
ولتاژ RMS	A	$0.00055963 * 1.017$	صفر
ولتاژ RMS	B	0.00055963	صفر
ولتاژ RMS	C	0.00055963، موقتاً کپی B	صفر
جریان RMS	A/B/C	$0.00036565 / 100$	صفر
توان اکتیو	A	$(0.0170276 / 10) * 1.02$	صفر
توان اکتیو	B/C	$0.0170276 / 10$	صفر
توان ظاهری	A/B/C	برابر ضریب اولیه توان اکتیو همان فاز	صفر
جریان نول	N	$0.00036565 / 100$ ، موقتاً کپی	صفر
جریان فاز			

این جدول عمداً تک‌تک کانال‌ها را نشان می‌دهد تا تغییر یک فاز به تغییر ناخواسته فاز دیگر منجر نشود. با وجود برابر بودن بعضی مقدارهای اولیه، هر خانه حافظه مستقل خود را دارد. تابع `App_Init` جدول را با `App_ResetCalibrationToDefaults` بارگذاری می‌کند. هر فراخوانی `App_SetPhaseCalibration` یا `App_CalibratePhaseMultiPoint` فقط خانه انتخاب‌شده را با منبع `APP_CALIBRATION_USER` جایگزین می‌کند. فراخوانی مجدد بازنشانی نرم‌افزاری کالیبراسیون، تمام ضرایب زمان اجرای حافظه را حذف می‌کند:

```
۱ /* Discard runtime coefficients and restore the full startup table. */
۲ App_ResetCalibrationToDefaults();
```

وجود ضریب جایگزین موقت برای فاز استفاده‌نشده هیچ ورودی را فعال یا خطر الکتریکی ایجاد نمی‌کند؛ فقط اگر بعداً سیگنال به آن فاز وصل شد، برنامه بلافاصله مقدار تقریبی منتشر می‌کند. پیش از اتکا، همان فاز مستقل

با مرجع کالیبره می‌شود.

۶.۱۲.۱۵ استفاده تک‌فاز، دو کانال مستقل و سه‌فاز

اندیس و رابط برنامه‌نویسی در هر سه حالت ثابت‌اند. محصول تک‌فاز می‌تواند فقط خانه ADE7880_PHASE_A را مصرف کند. دو مصرف‌کننده مستقل می‌توانند از A و B استفاده کنند و کالیبراسیون هر کدام جدا باشد. دستگاه سه‌فاز هر سه خانه A/B/C را پردازش می‌کند. نمونه تنظیم مستقل دو کانال:

```

۱ ADE7880_LinearCalibration current_a = {SCALE_A, OFFSET_A};
۲ ADE7880_LinearCalibration current_b = {SCALE_B, OFFSET_B};
۳
۴ (void)App_SetPhaseCalibration(APP_PHASE_QUANTITY_CURRENT_RMS,
۵                               ADE7880_PHASE_A, &current_a);
۶ (void)App_SetPhaseCalibration(APP_PHASE_QUANTITY_CURRENT_RMS,
۷                               ADE7880_PHASE_B, &current_b);
۸ /* Phase C keeps its startup default until explicitly replaced. */

```

در سطح کتابخانه قابل حمل نیز ADE7880_ReadPhase برای خواندن انتخابی یک فاز و ADE7880_ReadAll برای نمونه اتمیک A/B/C و نول موجود است. درایور هیچ نمونه سراسری از تراشه ندارد؛ بنابراین پروژه دیگر می‌تواند برای بیش از یک ADE7880، یک ADE7880_Device مستقل با رابط انتقال مستقل بسازد.

۷.۱۲.۱۵ تصمیم کالیبراسیون: یک خط چندنقطه‌ای، نه ضرایب قطعه‌ای

مسیر اندازه‌گیری ADE7880 ذاتاً خطی طراحی شده است. قرار دادن یک ضریب برای بار کم و ضریب دیگر برای بار زیاد می‌تواند در مرز دو بازه پرش بسازد و عیب واقعی offset، نویز یا خطای فاز را پنهان کند. روش نخست این پروژه یک رابطه $\text{engineering} = \text{raw} * \text{scale} + \text{offset}$ است که با کمترین مربعات روی همه نقاط ثبت‌شده برازش می‌شود. حداقل دو نقطه لازم است، سه نقطه برای کم/میانی/زیاد حداقل عملی و پنج نقطه برای ارزیابی بهتر خطی بودن توصیه می‌شود.

راهنمای رسمی AN-1171 حداکثر سه مرحله متفاوت را معرفی می‌کند: gain، phase و offset. این عبارت به معنی سه ضریب قطعه‌ای نیست. gain برای هر دستگاه لازم است؛ اصلاح فاز برای CT، به‌خصوص در ضریب توان پایین، اغلب لازم است؛ و offset وقتی دقت زیاد در دامنه دینامیکی وسیع خواسته شود اهمیت پیدا می‌کند. اگر پس از برازش خطی، خطای کم‌بار همچنان سیستماتیک باشد، باید رجیسترهای xIRMSOS/xVRMSOS/xWATTOS تراشه ارزیابی شوند.

۸.۱۲.۱۵ انتخاب نقاط و برداشت داده

در هر نقطه، پس از زمان نشست، دست‌کم ۲۰ تا ۶۰ نمونه خام ثبت و میانگین شوند. ابزار مرجع باید هم‌زمان خوانده شود. صفر کامل برای کالیبراسیون RMS مناسب نیست؛ راهنمای تراشه نیز offset را با یک سیگنال کم ولی غیرصفر اندازه می‌گیرد.

کمیت	نقاط پیشنهادی
RMS ولتاژ	پایین، نامی و بالای بازه واقعی؛ برای شبکه ۲۳۰ ولت، در صورت پشتیبانی منبع مثلاً ۱۸۰، ۲۳۰ و ۲۵۰ ولت.
RMS جریان	کمترین جریان موردنیاز معتبر، یک نقطه میانی و ۸۰ تا ۱۰۰ درصد جریان نامی طراحی؛ از حد مجاز CT/burden/ADE عبور نشود.
توان اکتیو	بارهای مرجع کم، میانی و زیاد با ولتاژ پایدار و PF معلوم؛ برای gain ساده، بار مقاومتی نزدیک $PF=1$.
توان ظاهری	هم‌زمان از ولتاژ و جریان مرجع یا پاورآنالایزر معتبر ثبت شود؛ صرف حاصل ضرب مقادیر نامطمئن استفاده نشود.

برای بررسی خطای فاز CT یک آزمون جدا با بار پایدار نزدیک $PF=0.5$ لازم است. نقاط برازش بهتر است کل بازه کاری را بپوشانند، ولی یک یا دو نقطه اضافی باید فقط برای اعتبارسنجی نگه داشته شوند و وارد برازش نشوند.

۹.۱۲.۱۵ کد کالیبراسیون ولتاژ، جریان و توان

در نمونه زیر نام‌های raw_... میانگین شمارهای ثبت‌شده از `g_app_electrical.raw.phase[0]` هستند و اعداد ستون دوم مقدار هم‌زمان ابزار مرجع در واحد مهندسی هستند:

```

۱  const ADE7880_CalibrationPoint voltage_points[] = {
۲      {raw_v_180, 180.0F}, {raw_v_230, 230.0F},
۳      {raw_v_250, 250.0F}
۴  };
۵  const ADE7880_CalibrationPoint current_points[] = {
۶      {raw_i_0_10, 0.10F}, {raw_i_1_00, 1.00F},
۷      {raw_i_5_00, 5.00F}
۸  };
۹  const ADE7880_CalibrationPoint power_points[] = {
۱۰     {raw_p_20, 20.0F}, {raw_p_200, 200.0F},
۱۱     {raw_p_1000, 1000.0F}
۱۲  };
۱۳  ADE7880_LinearCalibration voltage_result;
۱۴  ADE7880_LinearCalibration current_result;
۱۵  ADE7880_LinearCalibration power_result;
۱۶  float voltage_max_error_v;
۱۷  float current_max_error_a;
۱۸  float power_max_error_w;
۱۹
۲۰  (void)App_CalibratePhaseMultiPoint(
۲۱      APP_PHASE_QUANTITY_VOLTAGE_RMS, ADE7880_PHASE_A,
۲۲      voltage_points, 3U, &voltage_result, &voltage_max_error_v);
۲۳  (void)App_CalibratePhaseMultiPoint(
۲۴      APP_PHASE_QUANTITY_CURRENT_RMS, ADE7880_PHASE_A,
۲۵      current_points, 3U, &current_result, &current_max_error_a);
۲۶  (void)App_CalibratePhaseMultiPoint(
۲۷      APP_PHASE_QUANTITY_ACTIVE_POWER, ADE7880_PHASE_A,
۲۸      power_points, 3U, &power_result, &power_max_error_w);

```

سه مقدار `max_error` بیشترین فاصله نقاط ورودی از خط برازش‌شده را به‌ترتیب برحسب ولت، آمپر و وات نشان می‌دهند. کوچک‌بودن آن‌ها کافی نیست؛ خروجی باید در نقاط مستقل نیز با مرجع مقایسه شود. برای CT معکوس، مقدار مرجع همچنان خروجی نهایی مطلوب است و تابع کاربردی علامت ضریب توان اکتیو را با تنظیم `current_polarity` هماهنگ می‌کند.

نتیجه نصب‌شده تا `reset` در RAM است. پس از تأیید، اعضای `scale/offset` ساختارهای نتیجه در Watch

یادداشت و در Flash یا تنظیمات startup ذخیره شوند. نمونه بارگذاری ثابت تأییدشده:

```

۱ static const ADE7880_LinearCalibration saved_power_a = {
۲     .scale = VERIFIED_POWER_A_SCALE,
۳     .offset = VERIFIED_POWER_A_OFFSET
۴ };
۵
۶ (void)App_SetPhaseCalibration(
۷     APP_PHASE_QUANTITY_ACTIVE_POWER,
۸     ADE7880_PHASE_A, &saved_power_a);

```

جریان نول نیز مسیر چندنقطه‌ای همسان و مستقل دارد:

```

۱ const ADE7880_CalibrationPoint neutral_points[] = {
۲     {raw_n_0_10, 0.10F},
۳     {raw_n_1_00, 1.00F},
۴     {raw_n_5_00, 5.00F}
۵ };
۶ ADE7880_LinearCalibration neutral_result;
۷
۸ (void)App_CalibrateNeutralCurrentMultiPoint(
۹     neutral_points, 3U, &neutral_result, NULL);

```

توان نوشته‌شده روی بدنه لامپ فقط مقدار نامی است و جای وات‌متر یا منبع کالیبره را نمی‌گیرد. ترتیب حرفه‌ای پیشنهادی این است: جهت CT، بررسی فاز، کالیبراسیون gain چندنقطه‌ای، ارزیابی نقطه‌های مستقل، و فقط در صورت باقی‌ماندن خطای کم‌بار، اصلاح offset خود تراشه.

۱۳.۱۵ آزمون سخت‌افزاری مرحله به مرحله

به علت اتصال ورودی‌های ADE7880 به برق شهر، این آزمون فقط با ایزولاسیون مناسب، منبع محدودشده و فرد آگاه به ایمنی برق انجام شود. اتصال مستقیم اسیلوسکوپ یا دیباگر زمین‌شده به بخش غیرایزوله می‌تواند خطرناک و مخرب باشد.

۱.۱۳.۱۵ آزمون بدون اعمال ولتاژ اندازه‌گیری

۱. بارهای رله و ورودی‌های قدرت را ایمن یا جدا کنید.

۲. اتصال SWCLK=PA14، SWDIO=PA13، زمین مشترک و مرجع ۳.۳ ولت J-Link را برقرار کنید.
۳. برنامه را بسازید، پروگرام و reset کنید.
۴. اگر ADE7880 پاسخ دهد، LEDها باید الگوی رقص تقریباً یک‌ثانیه‌ای داشته باشند.
۵. اگر ارتباط برقرار نشود، دو LED باید سریع یکی‌درمیان شوند و communication_errors پس از تلاش‌های خواندن افزایش یابد.
۶. با logic analyzer کنترل کنید CS در هر تراکنش پایین، SCLK در بی‌کاری بالا و نرخ آن تقریباً ۵.۵۶۲ کیلوهرتز باشد.
۷. فرمان خواندن باید با 0x01 و فرمان نوشتن با 0x00 آغاز شود و سپس نشانی ۱۶ بیتی MSB-first بیاید.

۲.۱۳.۱۵ آزمون با منبع مرجع

۱. ابتدا ولتاژ کم و ایمن مطابق طراحی ورودی اعمال کنید و تغییر رجیستر متناظر را ببینید.
۲. فقط پس از تأیید سیم‌بندی، منبع AC مرجع و بار کنترل‌شده وصل شود.
۳. فازهای A، B و C جداگانه تغییر داده شوند تا نداشت هر سه کانال قطعی شود.
۴. علامت توان اکتیو با جهت توان کنترل شود؛ برای بار مصرفی باید مثبت باشد. اگر اصلاح فیزیکی ممکن نیست، App_SetCurrentPolarity یک‌بار تنظیم شود.
۵. ضریب توان یک بار مقاومتی باید به مثبت یک نزدیک باشد. برای بار غیرخطی، power_factor_abs بزرگی و power_factor اطلاعات علامت پیش‌فاز/پس‌فاز را نشان می‌دهد.
۶. قطع موقت سیم SPI یا تغذیه ADE باید حالت خطا ایجاد کند و پس از رفع عیب، تلاش دوره‌ای باید ارتباط را بازیابی کند.
۷. آزمون پیوسته حداقل ۲۴ ساعت برای شمار خطا، پایداری snapshot و الگوی LED انجام شود؛ آزمون چندروزه و چرخه دما برای ادعای صنعتی لازم است.

۱۴.۱۵ راهنمای عیب‌یابی

نشانه	بررسی پیشنهادی
LED خطا از ابتدای روشن شدن	تغذیه و کلاک کریستال ADE، اتصال PM0/PM1، سطح RESET، چهار خط SPI، سطح CS و مشترک بودن زمین بررسی شود.
تمام خواندن‌ها 0xFFFFFFFF	مسیر MISO باز، CS غیرفعال، تراشه بی‌تغذیه یا رابط هنوز روی I2C است.
تمام خواندن‌ها صفر	کلاک و ورودی‌ها، RUN، اتصال کوتاه MISO یا reset دائمی بررسی شود.
خطای متناوب	طول سیم، نویز، زمین، تغذیه، خازن‌های بای‌پس و شکل موج SCLK/CS با logic analyzer بررسی شود.
PF درست ولی ولت/آمپر نادرست	ابتدا داده raw را کنترل و سپس نسبت ورودی و ضرایب کالیبراسیون را اصلاح کنید.
توان مصرفی منفی است	جهت CT را در حالت بی‌برق با علامت‌های سازنده و شماتیک تطبیق دهید یا جبران نرم‌افزاری REVERSED را یکبار اعمال کنید.
توان مثبت ولی PF منفی است	الزاماً خطا نیست؛ PF علامت‌دار ADE7880 پیش‌فاز/پس‌فاز را مشخص می‌کند. بزرگی را از power_factor_abs بخوانید و با پاورآنالایزر مرجع مقایسه کنید.
ناشناخته‌بودن نوع DMA در Keil	فعال‌بودن ماژول DMA در تنظیمات HAL و عضویت سورس آن در پروژه بررسی شود.
پس از CubeMX کد کامپایل نمی‌شود	عضویت فایل‌های برنامه و کتابخانه، مسیرهای include و تعریف ADE_CS بررسی شود.

در همین خطا وجود فراخوانی `App_Init(&hspi2)` در بلوک کاربر فایل اصلی نیز کنترل شود.

۱۵.۱۵ نتیجه اعتبارسنجی نرم‌افزاری این مرحله

ساخت تازه با PlatformIO ststm32 20.0.0 و framework-stm32cubef1 1.8.7 با گزینه‌های -Wall -Wextra -Werror موفق شد. خروجی ۱۰۳۷۲ بایت Flash و ۵۴۴ بایت RAM مصرف کرد. ساخت دوم با

Keil و Arm Compiler 6.24 نیز با 0 Error(s), 0 Warning(s) پایان یافت و اندازه‌های Code=9848, RO-data=580, RW-data=12 و ZI-data=1532 بایت را گزارش کرد. فایل‌های اجرایی هر دو زنجیره‌ابزار تولید شدند. جدول پیش‌فرض A/B/C و نول، بازنشانی ضرایب و کالیبراسیون چندنقطه‌ای نول در هر دو زنجیره‌ابزار ساخته شدند. در ممیزی نهایی، تغییر ضریب یا جهت ترانس جریان نیز اصلاح شد تا مقدار مهندسی و نمایش قدیمی همان لحظه پاک شوند و دیباگر تا نمونه بعدی داده کهنه نشان ندهد. کاربر پیش‌تر برقراری ارتباط، دیباگ Keil، مشاهده ولتاژ و پاسخ جریان/توان یک بار لامپی روی برد واقعی را گزارش کرده است. دقت ضرایب جایگزین موقت فاز C و نول و دقت نهایی همه کانال‌ها هنوز باید با ابزار مرجع و نقاط مستقل روی برد کنترل شود.

۱۶.۱۵ چک‌لیست پذیرش این قابلیت

این مرحله زمانی از نظر سخت‌افزاری پذیرفته می‌شود که همه موارد زیر ثبت شوند:

- build بدون warning در Keil و PlatformIO؛
- تولید مجدد CubeMX و باقی‌ماندن منطق Application/Library؛
- مشاهده کلاک، CS و قالب فرمان صحیح با logic analyzer؛
- online=true و افزایش پیوسته شمار نمونه؛
- عدم افزایش خطای ارتباط در آزمون طولانی؛
- صحت نگاشت فازهای A، B و C و کانال نول؛
- کالیبراسیون هر کانال با مرجع معتبر و ثبت ضرایب؛
- بازیابی خودکار پس از قطع و وصل کنترل‌شده ارتباط یا تغذیه؛
- ثبت نتیجه، تاریخ، نسخه commit و تجهیزات آزمون در پرونده پروژه.

فصل ۱۶

زیرسامانه DS18B20 با شماره گذاری پایدار

۱.۱۶ نیازمندی و مسئله ترتیب

هر حسگر DS18B20 یک شناسه یکتای ۶۴ بیتی دارد. دستور Search ROM همه شناسه های متصل را پیدا می کند، اما ترتیب جست و جو همان شماره ای نیست که روی بدنه حسگر نوشته شده است. اضافه شدن، حذف یا تعویض یک شناسه می تواند ترتیب کشف را تغییر دهد. بنابراین مرتب سازی نتیجه جست و جو به تنهایی برای حفظ معنای «سنسور ۱»، «سنسور ۲» و مانند آن کافی نیست.

راه حل این نسخه جداسازی هویت فیزیکی از شماره منطقی است. برنامه برای هر شماره یک زوج «شماره گذرگاه و ROM» در Flash ذخیره می کند. در نتیجه خاموشی برد ترتیب را عوض نمی کند و نتیجه هر جست و جوی تازه فقط حضور یا غیبت همان هویت های ذخیره شده را تعیین می کند.

شماره کاربردی	گذرگاه	هویت ذخیره شده
۱	صفر، PB10	ROM یکتای حسگر برچسب ۱
۲	یک، PC7	ROM یکتای حسگر برچسب ۲
۳ تا ۳۲	صفر یا یک	در صورت معرفی کاربر

ظرفیت نرم افزار ۳۲ شماره و چهار گذرگاه قابل پیکربندی است؛ کاربرد فعلی دو گذرگاه دارد و لازم نیست همه جایگاه ها پر شوند.

۲.۱۶ اتصال الکتریکی و فرض‌های این نسخه

گذرگاه صفر به PB10 و گذرگاه یک به PC7 متصل است. فایل STM32F107.ioc هر دو را خروجی open-drain با سطح اولیه آزاد ثبت می‌کند. اتصال پشتیبانی‌شده سه‌سیمه است: VDD به تغذیه مجاز سنسور، GND به زمین مشترک و DQ به پایه گذرگاه. هر گذرگاه باید بالاکش خارجی به ولتاژ منطقی داشته باشد. مقدار متداول ۷.۴ کیلو اهم نقطه شروع است، نه تضمین برای هر طول کابل و تعداد حسگر؛ کیفیت لبه‌ها باید روی سخت‌افزار نهایی دیده شود.

حالت تغذیه انگلی یا parasite power به کلید strong pull-up نیاز دارد و در adapter فعلی پشتیبانی نشده است. نرم‌افزار بر تغذیه خارجی تکیه می‌کند. کابل بلند، توپولوژی ستاره‌ای، زمین نامناسب و نویز رله‌ها می‌توانند خطای CRC بسازند؛ موفقیت کامپایل جای مشاهده شکل موج و آزمون کابل واقعی را نمی‌گیرد.

۳.۱۶ معماری قابل انتقال

زیرسامانه به چهار مرز روشن تقسیم شده است:

۱. Libraries/OneWire هیچ هدر HAL ندارد. عملیات خط، تأخیر میکروثانیه و ورود یا خروج از ناحیه بحرانی را از callback می‌گیرد و reset، خواندن و نوشتن بیت و بایت، CRC8 و الگوریتم Search ROM را پیاده می‌کند.

۲. Libraries/DS18B20 فرمان‌های خانواده 0x28 شامل Match ROM، Skip ROM، Convert T، خواندن و نوشتن scratchpad، وضوح ۹ تا ۱۲ بیت و تبدیل مقدار علامت‌دار را فراهم می‌کند.

۳. Libraries/DS18B20Manager زمان، ذخیره‌سازی و آرایه گذرگاه‌ها را از پیکربندی دریافت می‌کند. این لایه کشف، نگاشت پایدار، تعویض کنترل‌شده و ماشین حالت خواندن را اداره می‌کند و تخصیص حافظه پویا ندارد. رفتار امن «ثبت تازه» و «تعویض بدون ابهام» پیش‌فرض ثابت مدیر است و دیگر دو پرچم پیکربندی جدا ندارد.

۴. Application/Src/app.c مرز برد حاضر است. PB10 و PC7، شمارنده سیکل Cortex-M3، HAL_GetTick و ذخیره‌ساز Flash فقط در این فایل به لایه‌های قابل حمل وصل می‌شوند. نمونه مدیر خصوصی است و هدر ساده برنامه فقط داده قابل مصرف را نشان می‌دهد.

برای انتقال کتابخانه به میکروکنترلر دیگر، سه لایه نخست تغییر نمی‌کنند. توسعه‌دهنده عملیات خط، تأخیر میکروثانیه و حفاظت کوتاه زمان‌بندی را پیاده می‌کند.

callbackهای ذخیره‌سازی را می‌توان به EEPROM، FRAM یا Flash مقصد متصل کرد.

۴.۱۶ جستوجو و تضمین صحت داده

الگوریتم جستوجو اختلاف بیت‌ها را طبق پروتکل 1-Wire دنبال می‌کند و حالت جستوجو برای هر گذرگاه مستقل است. فقط ROMهایی پذیرفته می‌شوند که بایت خانواده آن‌ها 0x28 و CRC8 هفت بایت اول با بایت هشتم یکسان باشد. فهرست کشف‌شده برای نمایش قطعی ابتدا با شماره گذرگاه و سپس بایت‌های ROM مرتب می‌شود؛ با این حال این مرتب‌سازی هرگز جای جدول شماره منطقی را نمی‌گیرد.

در خواندن دما، نه بایت scratchpad دریافت و CRC8 هشت بایت اول با بایت نهم مقایسه می‌شود. دو بایت دما به یک عدد علامت‌دار ۱۶ بیتی با واحد یک‌شانزدهم درجه سلسیوس تبدیل می‌شوند؛ بنابراین دماهای زیر صفر نیز درست می‌مانند. خروجی سلسیوس فقط پس از CRC صحیح و کنترل بازه منفی ۵۵ تا مثبت ۱۲۵ درجه معتبر اعلام می‌شود.

۵.۱۶ ماشین حالت و زمان‌بندی

مدیر از سه حالت IDLE، CONVERTING و READING استفاده می‌کند. در زمان نمونه‌برداری، روی هر گذرگاه یک Convert T سراسری اجرا می‌شود تا همه سنسورها هم‌زمان تبدیل را آغاز کنند. برنامه برای وضوح نامعلوم یا ۱۲ بیت، ۷۵۰ میلی‌ثانیه تا موعد تبدیل صبر می‌کند، ولی این انتظار با مقایسه زمان در App_Process است و حلقه اصلی متوقف نمی‌شود. سپس در هر اجرای حلقه حداکثر یک جایگاه آدرس‌دار خوانده می‌شود.

تابع DS18B20_ManagerProcess فقط زمانی true برمی‌گرداند که کشف یا وضعیت یک جایگاه تغییر کرده باشد. در نتیجه adapter آرایه‌های عمومی را فقط هنگام داده تازه بازنویسی می‌کند؛ پیمایش ۳۲ جایگاه در هر دور بی‌تغییر حلقه حذف شده است.

slotهای خود پروتکل در مقیاس میکروثانیه‌اند و برای جلوگیری از شکسته‌شدن زمان‌بندی، callback ناحیه بحرانی وضعیت وقفه را حفظ و بازیابی می‌کند. این حفاظت کوتاه با انتظار ۷۵۰ میلی‌ثانیه‌ای تفاوت دارد. جستوجوی نخست بلافاصله و جستوجوهای بعدی هر پنج ثانیه اجرا می‌شوند؛ در محصولی با محدودیت latency سخت‌گیرانه‌تر باید بودجه زمانی آن اندازه‌گیری شود.

۶.۱۶ ذخیره نگاشت در Flash

آخرین صفحه دو کیلوبایتی Flash از آدرس 0x0803F800 برای داده رزرو شده و linker تا پیش از آن کد را قرار می‌دهد. رکورد نسخه ۲ شامل magic، نسخه، اندازه payload، آرایه نگاشت و CRC32 است. بارگذاری فقط هنگامی موفق است که همه این فیلدها سازگار باشند. ROM نامعتبر، شماره گذرگاه خارج از محدوده و

نگاشت تکراری هنگام راه اندازی رد می شوند.

ثبت خودکار نخستین حضور، تعویض صریح یا پاک کردن شماره یک erase/program انجام می دهد. این نوشتن فقط هنگام تغییر جدول است و در نمونه برداری عادی تکرار نمی شود. نوشتن در Flash داخل وقفه انجام نمی شود.

پس از برنامه ریزی، رکورد با حافظه مقایسه می شود و در شکست، وضعیت RAM به نسخه قبلی برمی گردد. برای تحمل قطع تغذیه هنگام نوشتن، دو اسلات journal شده یا حافظه غیر فرار جداگانه گام مقاوم سازی بعدی است.

فرمت قدیمی فقط ROMها را داشت و شماره گذرگاه را ذخیره نمی کرد. تابع خواندن آن برای مهاجرت دستی باقی مانده، اما مهاجرت خودکار انجام نمی شود؛ نسبت دادن همه شناسه های قدیمی به یک گذرگاه می تواند نگاشت نادرست و پنهان بسازد.

۷.۱۶ معرفی اولیه سنسورها

استفاده عادی هیچ فراخوانی دستی ندارد. برنامه در نخستین جست و جو هر ROM تازه را به اولین شماره خالی متصل، نگاشت را ذخیره و تبدیل دما را آغاز می کند. برای یک حسگر کافی است آن را وصل و برد را روشن کرد؛ دما پس از حدود یک ثانیه در `g_temperature_c[0]` ظاهر و `g_temperature_valid[0]` درست می شود.

برای تطبیق شماره های نرم افزاری با برچسب روی بدنه، حسگرها یک به یک و به ترتیب متصل شوند:

۱. فقط حسگر دارای برچسب ۱ به گذرگاه موردنظر وصل شود.

۲. تا درست شدن `g_temperature_valid[0]` صبر شود؛ تابعی از دیباگر صدا زده نمی شود.

۳. حسگر برچسب ۲ وصل و تا درست شدن `g_temperature_valid[1]` صبر شود.

۴. همین روند برای تعداد موردنیاز تکرار شود. `g_temperature_sensor_count` تعداد هویت های ذخیره شده را نشان می دهد.

اگر همه حسگرها در نخستین بوت همزمان حاضر باشند، شماره ها با ترتیب قطعی «شماره گذرگاه، سپس ROM» تخصیص می یابند. این ترتیب پس از ذخیره ثابت است، ولی الزاماً با نوشته روی بدنه یکی نیست؛ پروتکل اطلاعاتی درباره برچسب انسانی ندارد. APIهای تخصیص صریح فقط برای ابزار تولید و تعمیر پیشرفته باقی مانده اند.

۸.۱۶ تعویض سنسور و مرز تصمیم خودکار

سنسور جایگزین ROM تازه دارد و پروتکل راهی برای فهمیدن برچسب فیزیکی آن ارائه نمی‌کند. تعویض خودکار فقط وقتی امن است که روی یک گذرگاه دقیقاً یک نگاشت ذخیره‌شده غایب و دقیقاً یک ROM جدید حاضر باشد.

در آن حالت ROM جدید در همان شماره نوشته و شمارنده automatic_replacements زیاد می‌شود.

اگر دو حسگر غایب یا دو حسگر تازه حاضر باشند، مسئله از دید نرم‌افزار مبهم است. مدیر عمداً هیچ حدسی نمی‌زند. در مسیر عادی سنسورها باید یکی‌یکی و در وضعیت یک غایب/یک تازه تعویض شوند. ابزار سرویس می‌تواند مستقیماً از DS18B20_ManagerAssignDiscovered استفاده کند، اما این API پیشرفته عمداً در هدر ساده برنامه منتشر نشده است. این رفتار از جابه‌جایی خاموش نقش‌های کنترلی جلوگیری می‌کند.

۹.۱۶ مشاهده در دیباگر

برای شماره N مسیر ساده دما چنین است:

```
1 g_temperature_c[N - 1]
```

اعتبار مقدار از مسیر زیر دیده می‌شود:

```
1 g_temperature_valid[N - 1]
```

تابع App_TemperatureGetCelsius نیز همین کنترل را انجام می‌دهد و فقط برای نمونه معتبر true برمی‌گرداند. این تنها تابع عمومی لایه کاربرد است. ساختار مدیر خصوصی شده تا برنامه عادی به ROM، گذرگاه، وضعیت داخلی یا API‌های تعمیراتی وابسته نشود.

برای پروژه‌ای با دقیقاً یک سنسور و جایی که توقف کوتاه پذیرفتنی است، حتی مدیر لازم نیست. پس از مقداردهی OneWire_Bus می‌توان تابع DS18B20_ReadTemperatureBlocking را با ROM=NULL و مهلت ۷۵۰ میلی‌ثانیه فراخوانی کرد. برای چند سنسور، شماره پایدار یا اجرای غیرمسدودکننده، مدیر انتخاب درست است.

۱۰.۱۶ حفاظت در برابر تولید مجدد CubeMX

تنظیم پایه‌ها در فایل IOC ذخیره شده و منطق در Application و Libraries بیرون فایل‌های مولد قرار دارد. فایل مولد فقط در بلوک‌های USER CODE توابع App_Init و App_Process را فراخوانی می‌کند. پس از

هر Generate Code باید diff گیت، حالت تخلیه باز PB10/PC7، فراخوانی های USER CODE، مسیرهای include و عضویت سورس مدیر در پروژه Keil بررسی شوند.

PlatformIO همه سورس های کتابخانه را با الگوی پوشه ها می سازد و مسیر include جدید در platformio.ini ثبت شده است. فایل STM32F107.uvprojx نیز سورس مدیر و مسیر هدر آن را دارد. این همگام سازی باعث می شود یک کد C در هر دو زنجیره ساخت استفاده شود، نه دو نسخه جدا.

۱۱.۱۶ چک لیست آزمون روی برد

۱. ابتدا یک حسگر روی هر گذرگاه، سپس تعداد واقعی سنسورها آزمایش شود؛ شکل موج reset، presence و slot در دورترین نقطه کابل دیده شود.

۲. ROM، شماره گذرگاه، CRC، دمای مثبت و منفی و مقایسه با دماسنج مرجع ثبت شود.

۳. پس از معرفی شماره ها، خاموشی کامل و روشن شدن دوباره انجام و ثبات slots تأیید شود.

۴. یک حسگر قطع و دوباره وصل شود؛ پرچم های present/valid و شمارنده خطا بررسی شوند.

۵. سناریوی یک خراب و یک تازه روی همان گذرگاه، سپس سناریوی مبهم دو خراب و دو تازه آزموده شود. اولی باید خودکار و دومی فقط صریح جایگزین شود.

۶. نویز رله، طول کابل نهایی، دمای ۸۵ درجه تکراری، خطای CRC، قطع تغذیه هنگام ذخیره و اجرای پیوسته حداقل ۲۴ ساعت بررسی شود.

۷. ساخت واقعی Keil و PlatformIO بعد از هر تولید مجدد CubeMX تکرار شود.

۱۲.۱۶ محدودیت های اعلام شده

اعتبارسنجی فعلی شامل بررسی دیتاشیت، بازبینی کد و ساخت فریمور است. هیچ نتیجه ای درباره کیفیت pull-up، کابل، نویز، دقت دما یا رفتار سنسور واقعی بدون آزمون برد ادعا نمی شود. همچنین دماها هنوز به قاب سریال قدیمی یا منطق کنترل محصول متصل نشده اند؛ این جداسازی عمدی است تا ابتدا هویت و خواندن سنسورها مستقل تأیید شود.

فصل ۱۷

نمایشگر کاراکتری و کلیدهای منو

۱.۱۷ هدف و نتیجه این مرحله

در این مرحله نمایشگر ۱۶ ستون و ۲ سطر مدل LCD-016N002B-CFH-ET و سه کلید افزاینده، کاهنده و صفرکننده به مسیر اجرایی اضافه شدند. نمایشگر با گذرگاه چهار بیتی کار می‌کند، هر فشار معتبر کلید دقیقاً یک رخداد می‌سازد و مقدار روی بازه بسته صفر تا صد محدود می‌شود. حلقه اصلی همچنان سنجش انرژی، دما و الگوی دو چراغ تشخیصی را سرویس می‌کند.

نمایش اولیه چنین است:

```
1 Counter test
2 Value:    0 /100
```

فشار کلید اول یک واحد اضافه، کلید دوم یک واحد کم و کلید سوم مقدار را صفر می‌کند. در صفر، کلید کاهنده اثری ندارد و در صد، کلید افزاینده اثری ندارد. فقط سه خانه عددی بازنویسی می‌شوند؛ نمایشگر در هر تغییر پاک نمی‌شود و بنابراین چشمک ناشی از فرمان Clear Display وجود ندارد.

۲.۱۷ مبنای دیتاشیت و محدودیت الکتریکی مهم

دیتاشیت ۲۶ صفحه‌ای Vishay در [datasheets/lcd016n002bcfh.pdf](https://www.vishay.com/datasheets/lcd016n002bcfh.pdf) بررسی شد. ماژول ۱۶ در ۲، کنترلر ST7066 یا معادل، فونت ۵ در ۸ و تغذیه منطقی نامی ۵ ولت دارد. سطر نخست از آدرس DDRAM 0x00 و سطر دوم از 0x40 آغاز می‌شود. زمان اجرای فرمان عادی حدود ۳۹ میکروثانیه، نوشتن داده حدود ۴۳ میکروثانیه و Clear/Home حدود ۱/۵۳ میلی‌ثانیه است. درایور برای حاشیه امن به ترتیب ۵۰ و ۱۶۰۰

میکروثانیه صبر می‌کند و پالس E یک میکروثانیه است که از حداقل ۱۴۰ نانو ثانیه بزرگ‌تر است.

هشدار: در تغذیه ۵ ولت، دیتاشیت حداقل سطح بالای ورودی را ۰.۷ VDD یعنی ۳/۵ ولت اعلام می‌کند. خروجی ۳/۳ ولتی STM32 این شرط تضمین‌شده را برآورده نمی‌کند، هرچند ممکن است روی یک نمونه کار کند. برای محصول قابل اتکا باید مبدل سطح مناسب روی RS/E/DB4..DB7 استفاده شود یا نسخه و شرایط تغذیه سازگار با منطق ۳/۳ ولت با دیتاشیت همان قطعه تأیید شود.

پایه R/W به PA7 و یک مقاومت پایین‌کش متصل است. نرم‌افزار این پایه را خروجی پوش‌پول در سطح پایین نگه می‌دارد و Busy Flag را نمی‌خواند. جهت DB4..DB7 همیشه خروجی می‌ماند و احتمال درگیری خروجی ۵ ولتی LCD با GPIO میکرو حذف می‌شود. پایین‌کش خارجی نیز هنگام بازنشانی مانع شناور شدن R/W می‌شود.

۳.۱۷ جدول پایه‌ها و تنظیم CubeMX

سیگنال	پایه STM32	حالت GPIO
E	PA6	خروجی پوش‌پول، سطح پایین اولیه
R/W	PA7	خروجی پوش‌پول، همیشه سطح پایین
RS	PA8	خروجی پوش‌پول
DB4..DB7	PA9..PA12	چهار خروجی پوش‌پول
افزایش	PD9 / EXTI9	وقفه لبه بالارونده، بدون مقاومت داخلی
کاهش	PD10 / EXTI10	وقفه لبه بالارونده، بدون مقاومت داخلی
صفر کردن	PD11 / EXTI11	وقفه لبه بالارونده، بدون مقاومت داخلی

کلیدها در سخت‌افزار پایین‌کش خارجی دارند و با فشار به تغذیه مثبت وصل می‌شوند؛ پس لبه بالارونده آغاز فشار است و GPIO_NOPULL انتخاب شده است. خطوط ۹ و ۱۰/۱۱ به ترتیب از بردارهای مشترک EXTI9_5 و EXTI15_10 استفاده می‌کنند. هر handler فقط تابع استاندارد HAL را برای پایه مربوط فراخوانی می‌کند.

همه نام‌ها، حالت پایه‌ها و فعال بودن NVIC در STM32F107.ioc ثبت شده‌اند. PA13/PA14 برای SWD باقی مانده‌اند. PA9/PA10 دیگر برای USART1 آزاد نیستند؛ اگر در آینده USART1 لازم شود باید LCD یا UART به پایه/واحد دیگری منتقل و IOC هم‌زمان اصلاح شود.

۴.۱۷ معماری نرم افزار

کتابخانه قابل حمل در Libraries/CharacterLcd هیچ هدر STM32 یا HAL را وارد نمی کند. ساختار CharacterLcd_Config پنج callback و هندسه نمایشگر را می گیرد:

○ write_rs: انتخاب فرمان یا داده؛

○ write_rw: اختیاری و در این برد فقط برای نگه داشتن سطح پایین؛

○ write_enable: ساخت پالس فعال سازی؛

○ write_data4: قراردادن نیم بایت روی DB4..DB7؛

○ delay_us: تأخیر کوتاه مستقل از نوع میکروکنترلر.

لایه برد و برنامه آزمایشی در Application/Src/ui_counter.c قرار دارد. این فایل callback ها را به GPIO های STM32 متصل، LCD را مقداردهی، جهش کلیدها را حذف و مقدار آزمایشی را نمایش می دهد. app.c فقط یک بار UiCounter_Init و در حلقه UiCounter_Process را فراخوانی می کند. متغیر g_ui_counter_value برای مشاهده مستقیم در Watch در دسترس است.

۵.۱۷ راه اندازی چهار بیتی و زمان بندی

پس از پایدار شدن تغذیه، درایور ۵۰ میلی ثانیه صبر می کند. سپس RS=0 و R/W=0 شده، نیم بایت های شروع چهار بیتی ارسال می شوند و فرمان 0x28 رابط چهار بیتی، دو سطر و فونت ۵ در ۸ را انتخاب می کند. بعد نمایش موقتاً خاموش، DDRAM پاک، جهت حرکت مکان نما رو به راست و نمایش بدون cursor/blink روشن می شود.

هر بایت ابتدا نیم بایت بالا و سپس پایین را می فرستد. callback داده در برد حاضر هر چهار خط را با یک نوشتن رجیستر BSRR عوض می کند. تمام تأخیرهای میکروثانیه از DWT_CYCCNT در فرکانس ۷۲ مگاهرتز می آیند. تنها انتظار نسبتاً بلند، ۵۰ میلی ثانیه راه اندازی یک باره هنگام بوت است؛ مسیر عادی کلیدها فاقد HAL_Delay است.

۶.۱۷ مرجع توابع کتابخانه

تابع	کاربرد
CharacterLcd_Init	اعتبارسنجی callbackها و هندسه و راهاندازی کنترلر؛ در موفقیت true.
CharacterLcd_Clear	پاک کردن DDRAM و رفتن به سطر/ستون صفر؛ برای نوسازی مکرر توصیه نمی‌شود.
CharacterLcd_Home	بازگشت مکان‌نما بدون پاک کردن متن.
CharacterLcd_SetCursor	انتخاب ستون و سطر صفرمبنا؛ برای مختصات خارج صفحه false.
CharacterLcd_PutChar	نوشتن یک کد؛ نویسه خط جدید به ابتدای سطر بعد می‌رود و انتهای سطر شکسته می‌شود.
CharacterLcd_Print	نوشتن رشته C خاتمه‌یافته با صفر.
CharacterLcd_PrintUInt32	تبدیل و نمایش عدد بدون sprintf.
CharacterLcd_PrintInt32	نمایش عدد علامت‌دار و مدیریت امن INT32_MIN.
CharacterLcd_SetDisplay	کنترل نمایش، cursor و blink بدون حذف DDRAM.
CharacterLcd_Shift	جابجایی پنجره نمایش یک خانه.
CharacterLcd_CreateChar	نوشتن الگوی ۵ در ۸ در یکی از هشت جایگاه CGRAM و بازگرداندن مکان‌نما.

نمونه ساده:

```

1 CharacterLcd_Print(&lcd, "Voltage:");
2 CharacterLcd_SetCursor(&lcd, 0, 1);
3 CharacterLcd_PrintUInt32(&lcd, 230);
4 CharacterLcd_Print(&lcd, " V");

```

نمونه کاراکتر سفارشی:

```

1 static const uint8_t bell[8] = {
2     0x04, 0x0E, 0x0E, 0x0E, 0x1F, 0x00, 0x04, 0x00
3 };
4 CharacterLcd_CreateChar(&lcd, 0, bell);
5 CharacterLcd_PutChar(&lcd, 0);

```

مدل ET جدول کاراکتر انگلیسی و اروپایی دارد؛ LCD کاراکتری Unicode و متن فارسی را مانند نمایشگر گرافیکی پشتیبانی نمی‌کند. تنها هشت شکل ۵ در ۸ را می‌توان هم‌زمان در CGRAM تعریف کرد. بنابراین متن رابط فعلی عمداً ASCII است.

۷.۱۷ وقفه کوتاه و حذف جهش کلید

تابع callback وقفه HAL فقط یک بیت در `button_irq_mask` ثبت می‌کند. داخل ISR هیچ نوشتن LCD، تأخیر، تبدیل رشته، حلقه حذف جهش یا محاسبه برنامه انجام نمی‌شود. این پرچم در حلقه اصلی به‌صورت کوتاه و اتمیک برداشته می‌شود.

برای هر کلید یک حالت فشار و دو نامزد فشار/رهاسازی نگهداری می‌شود. پس از لبه بالارونده، پایه باید ۳۰ میلی‌ثانیه پیوسته در سطح بالا بماند تا فقط یک رخداد ساخته شود. سپس تا زمانی که پایه ۳۰ میلی‌ثانیه پیوسته در سطح پایین نشده، همه لبه‌های ناشی از جهش یا نگهداشتن کلید نادیده گرفته می‌شوند. نتیجه این است که نگهداشتن کلید تکرار خودکار ندارد و هر چرخه کامل فشار/رهاسازی حداکثر یک واحد تغییر می‌دهد. این طراحی از سرریز شمارنده میلی‌ثانیه آسیب نمی‌بیند، زیرا زمان به‌صورت تفریق بدون علامت مقایسه می‌شود. حدگذاری پیش از افزایش/کاهش انجام می‌شود و نوع ۸ بیتی هرگز از صد عبور یا زیر صفر باز نمی‌گردد.

۸.۱۷ حفظ LED و کارهای هم‌زمان

در هر دور `App_Process` ابتدا `DiagnosticLed_Process` و سپس `UiCounter_Process` اجرا می‌شوند و بعد ماشین حالت دما و نمونه‌برداری ADE7880 ادامه می‌یابد. الگوی LED با شمارنده زمان جلو می‌رود و برای LCD یا حذف جهش متوقف نمی‌شود. تغییر شمارنده فقط سه کاراکتر می‌نویسد و فرمان `Clear` در مسیر فشار کلید وجود ندارد.

وقفه‌های کلید با اولویت ۵ فعال‌اند. بازه‌های کوتاه 1-Wire ناحیه بحرانی خود را دارند؛ اگر لبه کلید دقیقاً در آن بازه برسد، سخت‌افزار EXTI آن را معلق نگه می‌دارد و پس از بازشدن وقفه ثبت می‌شود. حذف جهش سی میلی‌ثانیه‌ای تأخیر چند میکروثانیه‌ای را بی‌اثر می‌کند.

۹.۱۷ انتقال به میکروکنترلر دیگر

فقط پوشه `Libraries/CharacterLcd` به پروژه مقصد منتقل می‌شود. فایل سورس به ساخت و پوشه `Inc` به مسیر هدر افزوده و callback های GPIO و تأخیر مقصد نوشته می‌شوند:

```

1 CharacterLcd_Config config = {
2     .context = &board,
3     .write_rs = board_rs,
4     .write_rw = board_rw,          /* NULL if R/W is tied low */
5     .write_enable = board_e,
6     .write_data4 = board_data,
7     .delay_us = board_delay_us,
8     .columns = 16,
9     .rows = 2
10 };
11 CharacterLcd_Init(&lcd, &config);

```

هیچ شماره پایه، رجیستر STM32 یا نمونه سراسری برد در کتابخانه وجود ندارد. برای LCD چهارسطری، جابه‌جایی‌های رایج 0x00,0x40,0x14,0x54 پشتیبانی شده‌اند؛ ماژولی با نگاشت متفاوت به توسعه کوچک پیکربندی آدرس نیاز دارد.

۱۰.۱۷ حفاظت در برابر Generate Code

منطق LCD و حذف جهش بیرون فایل‌های مولد، در Libraries و Application است. فراخوانی‌های اصلی از همان App_Init/App_Process موجود در بلوک‌های USER CODE عبور می‌کنند. پایه‌ها و NVIC در IOC ثبت شده‌اند تا CubeMX بتواند main.h، MX_GPIO_Init و handlerها را دوباره بسازد.

پس از هر Generate Code باید تفاوت گیت کنترل شود، زیرا پروژه حاضر پیش از این مرحله نیز بخش‌هایی دست‌نویس در فایل‌های مولد داشته است. وجود پایه‌های PA6..PA12 و PD9..PD11، دو IRQ، فراخوانی‌های برنامه، مسیر هدر کتابخانه و عضویت دو فایل تازه در Keil باید بررسی شود. فایل‌های platformio.ini و STM32F107.uvprojx هر دو همگام شده‌اند.

۱۱.۱۷ روش آزمون روی برد

۱. پیش از پروگرام، مشترک بودن زمین، اتصال کنتراست، محدودکننده جریان نور پس‌زمینه و پایین ماندن R/W کنترل شود. سطح منطقی بالا نیز با راهکار مبدل سطح تأیید شود.

۲. پس از روشن شدن، دو متن ثابت و مقدار صفر باید بدون خانه‌های سیاه یا کاراکتر جابه‌جا دیده شوند. اگر

- فقط خانه سیاه است، ابتدا کنتراست و سپس توالی E/RS/DB4..DB7 بررسی شود.
۳. کلید افزایش یکبار فشرده و رها شود؛ مقدار باید دقیقاً یک شود. نگه داشتن کلید حداقل دو ثانیه نباید تغییر دیگری بسازد.
۴. هر کلید ۵۰ بار با سرعت‌های مختلف آزموده شود؛ هیچ پرش دوواحدی یا فشار گم‌شده پس از رهاسازی پایدار مجاز نیست.
۵. مقدار به صد رسانده و ده بار دیگر افزایش داده شود؛ باید صد بماند. سپس تا صفر کاهش و ده فشار اضافه امتحان شود؛ باید صفر بماند.
۶. در مقادیر میانی کلید بازنشانی آزموده شود. بازنشانی در صفر نیز نباید چشمک یا پاک شدن صفحه بسازد.
۷. هم‌زمان دو LED، g_app_electrical.updated_at_ms و داده دما دیده شوند تا توقف سرویس‌های قبلی رد شود.
۸. آزمون چندساعته در حضور نویز رله انجام و شکل موج تغذیه، E و کلیدها ثبت شود. ساخت موفق جای این پذیرش سخت‌افزاری را نمی‌گیرد.

۱۲.۱۷ عیب‌یابی و اعتبارسنجی

اگر نمایشگر خالی است، ترتیب بررسی تغذیه و کنتراست، زمین مشترک، سطح منطقی بالا، اتصال R/W، نام پایه‌ها و پالس E است. نمایش نیم‌بایت‌ها یا کاراکترهای بی‌معنا معمولاً از جابه‌جایی DB4..DB7 یا شروع نامعتبر چهار بیتی می‌آید. اگر کلید کار نمی‌کند، سطح آزاد باید نزدیک صفر و سطح فشار نزدیک تغذیه باشد و بیت معلق رجیستر EXTI و handler مشترک کنترل شوند.

هر دو سورس تازه با کل پروژه و گزینه‌های Werror - Wextra - Wall در PlatformIO ساخته شدند. مصرف ساخت این مرحله ۱۶۵۶۰ بایت Flash و ۳۲۹۲ بایت RAM است. XML پروژه Keil و مسیرهای سورس/هدر همگام شده‌اند. این اعتبارسنجی قرارداد نوع‌ها، لینک IRQها و حضور فایل‌ها را ثابت می‌کند؛ رفتار واقعی کنتراست، سطح منطقی، ترتیب سیم‌ها و جهش فقط روی برد قابل تأیید است.